KOMODO: Optimal Gray-Box Database Monitoring for Snapshot Isolation

Lasse Møldrup*, Yifan Dong*, Giovanna Kobus Conrado*, and Andreas Pavlogiannis
Department of Computer Science
Aarhus University
Aarhus, Denmark
{moeldrup, yvan.dong, gkc, pavlogiannis}@cs.au.dk

***Abstract*—Snapshot Isolation (SI) is a standard isolation level implemented by several popular databases, as it enables high-throughput transaction processing combined with strong data integrity. Due to discoveries of bugs in the isolation level implementations of popular databases, considerable efforts have been made towards developing isolation testers. These typically work in either a black-box or a white-box manner, meaning that the database internals are opaque or transparent to the tester.**

In this work, we propose *gray-box monitoring* as a uniform and efficient approach to testing for SI, which is also more amenable to deployment in a production setting. The monitor observes, in real time, the stream of transactions that the database commits, which is often API-supported via mechanisms such as change data capture (CDC). Upon processing a transaction t, the monitor has to decide whether the whole stream up to t is consistent with SI. We develop a monitor that performs this task and runs in $O(n \cdot \alpha)$ time, for a transaction stream of size n, where α bounds the number of pairwise-concurrent transactions. Moreover, we show that this time is optimal under high congestion, based on standard complexity-theoretic hypotheses, but reduces to the (also optimal) $O(n)$ time during low congestion, where transactions are actually serializable.

Finally, we implement our monitor in a tool called KOMODO, and evaluate its performance on seven production databases. Our results show that KOMODO is lightweight: in practice, its running time is linear, its memory usage depends only on the number of database keys and not on the stream size, and it manages to keep pace with the client throughput. Using KOMODO, we found bugs in the SI implementation of two databases.

***Index Terms*—Isolation levels, system testing, monitoring**

I. INTRODUCTION

Database isolation is a rigorous contract between a database and its clients, which specifies the degree to which various transactions may observe concurrency when executing in parallel. The strongest, and conceptually simplest isolation level is Serializability (SER), which specifies that transactions effectively execute atomically, thereby allowing no interleaving between them. Although Serializability is the most natural and desired guarantee from a client's perspective, it poses too large of an efficiency overhead for the ever-increasing need for throughput in modern, large-scale databases. To mitigate this overhead, Snapshot Isolation (SI) [1] leverages multiversioning to provide each transaction with a consistent, point-in-time view of the database. This ensures that readers do not block writers, and writers do not block readers, enabling significant performance gains, with only a small impact on data integrity during high contention. As such, SI is implemented by many of the most popular databases, such as PostgreSQL [2], Oracle [3], and Microsoft SQL Server [4].

Unfortunately, it is well-known that databases often do not correctly implement their claimed isolation guarantees, with numerous subtle bugs found in their implementations [5]. This has motivated a line of work on *black-box* isolation testing [6], [7], [8], [9], [10], [11], [12], [13], [14], adhering to the following operating principle. First, a client generates a high database load and collects a history of the database responses. Then, the client analyzes the history to detect anomalies in the responses; in particular, that there is no possible order in which the database could commit the client's transactions that is consistent with the claimed isolation level. For the stronger isolation levels, i.e., those that guarantee reasonable data integrity, such as SER and SI, black-box testing is known to be computationally demanding, i.e., it is NP-complete [6], [15]. This primarily stems from the black-box treatment, where the client has only a partial view of the commit order that the database established.

Recent work has shown that *white-box* techniques can effectively be used to circumvent this NP-hardness [16], [17], [18]. Such approaches obtain fine-grained information about database execution by instrumenting the database, reading from log files, or using database-specific public APIs, as well as other, implementation-specific information, such as the start timestamp of each transaction. In contrast to black-box testers which are database-independent, white-box testers are database-specific by nature.

In this paper, we formulate isolation testing in a *monitoring* and *gray-box* setting. The *monitoring* aspect denotes that the tester does not control the client's interaction with the database; instead, it observes the database's interactions with client applications in real time. This allows the monitor to run in a live production environment, increasing the likelihood of detecting elusive bugs in real-world workloads while enabling

*These authors contributed equally to this work.

immediate, on-the-fly corrective actions. The *gray-box* aspect dictates that information is extracted from the database using only common, well-documented APIs, while still relying on some database-specific assumptions about the information gathered. Specifically, we design the monitor to require *only the commit order* of transactions from the database. This design choice is motivated by two key considerations. First, from an abstraction perspective, isolation levels are contracts between databases and their clients. The most elegant and conceptually clear formalizations of isolation are axiomatic, relying solely on a commit order and a visibility relation [6], [19]. Further artifacts, such as timestamps, lock acquisition graphs, or internal MVCC version identifiers, are database-specific implementation details that should not be factored into a generic verification of this contract. Second, extracting the commit order is natively supported by public database APIs—most notably via *change data capture* (CDC) mechanisms, which stream these events in a standardized way through platforms like Debezium. Consequently, we avoid intrusive and error-prone techniques, such as instrumenting the database engine or parsing undocumented internal log files.

Naturally, there is a pragmatic need for a monitor to be as lightweight as possible, since its performance translates to a time and memory overhead in deployment. *How efficiently can we monitor for* SI *in terms of algorithmic complexity? Are there provable lower bounds on the time and memory for this task? In practice, is such a monitor sufficiently lightweight to keep pace with the throughput of live production?* We address these questions in this work.

Our contributions. We make the following contributions.

(1) We formalize *gray-box monitoring* as a practical and rigorous paradigm for testing database isolation guarantees in production. Under this model, the internal database engine remains entirely opaque. The monitor relies solely on the transaction commit order, which is implementation-agnostic and captures the essence of the isolation contract.
(2) We develop a monitor for SI that runs in $O(n \cdot \alpha)$ time, where n is the size of the log-stream and α bounds the number of pairwise-concurrent transactions. We further show that this monitor is essentially time-optimal under the Hitting Set hypothesis [20], i.e., the $O(n \cdot \alpha)$ time cannot be improved in general. Moreover, our monitor runs in $O(n)$ time when the log-stream is serializable (which is the typical case).
(3) Although our monitor requires $O(n)$ memory in the worst case, we develop a simple heuristic that keeps memory usage low in practice. Moreover, we prove that, in theory, monitoring for SI requires the considerable $\Omega(\nu^\ell)$ memory in the worst case, where ℓ and ν are the respective number of keys and values in the log-stream. This formally justifies why low memory usage can only be achieved heuristically.
(4) We implement our monitor in a tool called KOMODO, together with a monitoring architecture based on CDC and Debezium, and evaluate it on seven production databases: MariaDB, MySQL, OceanBase, Oracle, Percona Server for MySQL, YugabyteDB, and TiDB. KOMODO runs in linear time in practice and keeps pace with the rate at which the database commits transactions, beyond 10,000 operations per second, while its memory stays proportional to the number of keys. We further evaluate the monitoring algorithm independently of the surrounding architecture, and compare its performance to the state-of-the-art SI checker CHRONOS. Our results show that KOMODO is clearly faster, despite not requiring the start timestamp of each transaction, in contrast to CHRONOS. Moreover, KOMODO has found bugs in the SI implementation of TiDB and OceanBase, that can cause the wrong value to be read in certain scenarios.

II. OVERVIEW

We first motivate our setting and illustrate the main ideas of the paper. The formal treatment starts in Section III.

Database log-streams. A database transaction consists of reads and writes on a set of keys; we write $\mathrm{W}(x, v)$ for a write of value v on key x, and $\mathrm{R}(x, v)$ for a read that returns v on x. Each transaction is associated with a *session*, corresponding to the client that issued it. A monitor observes the stream of transactions committed by the database, which we call a *log-stream* L; the order in which the transactions are streamed is the *commit order* co. The *session order* so is the restriction of co to transactions on the same session. Figure 1 shows an example log-stream L with two sessions, s_1 and s_2.

The gray-box monitoring problem. A log-stream does not expose the *write-read* relation wr matching each read to the write it observed, since a read returns a *value*, and not the identity of its writer. The commit order, in contrast, is visible, and it is a good interface to monitor through, as (i) it is a ubiquitous concept among databases, (ii) it is sufficient to define common isolation levels, and (iii) it can often be extracted in an opaque, implementation-agnostic way by means of mechanisms such as CDC.

The problem of *gray-box monitoring* (or simply *monitoring* in the context of this work) is to decide, upon processing each transaction t, whether L up to and including t satisfies a given isolation level.

Monitoring for Snapshot Isolation (SI). At a high level, SI requires that each transaction t must observe the state of the database at a given point in time. In contrast to SER, however, this point is not necessarily immediately prior to t, which makes the consistency checks non-trivial.

The *database state* at a transaction t' is a map from keys to values, capturing the effect of all updates to the database performed by transactions that committed up to and including t'. The *snapshot taken* by a transaction t is the database state that t observes. For example, the snapshot taken by t_2 in Fig. 1

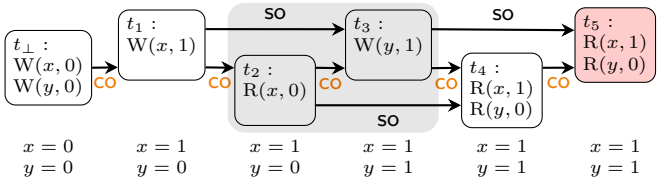


Fig. 1: A log-stream consisting of an initialization transaction $t_\perp$ and five transactions over keys x, y, shown in commit order co. Session s_1 consists of t_1, t_3, t_5, while session s_2 consists of t_2, t_4, with the corresponding transactions arriving in session order so. The database state after a transaction has committed is shown underneath the transaction. The shaded transactions $\{t_2, t_3\}$ are the candidates for the snapshot taken by t_4. A violation of SI occurs at t_5, due to its read $\mathrm{R}(y, 0)$.

can only be the database state at $t_\perp$, since $\mathrm{R}(x, 0)$ is observing 0 and thus misses the update on x by t_1.

In general, a transaction t cannot simply take a snapshot at any transaction. SI specifies that the snapshot must succeed (i) all transactions t' preceding t in the same session, and (ii) all transactions t' that committed before t and which wrote a key that t also writes. Therefore, the candidates for the snapshot taken by t form an interval $[t_{\text{MAX}}(t), t)$ of co, where $t_{\text{MAX}}(t)$ is the co-latest among all such transactions t'. In the example, the candidate snapshots for t_4 are the database states at t_2 and t_3, since t_2 is in the same session as t_4. Between these two, only the state at t_2 is consistent with the values that t_4 observes (since t_3 updates y to 1), and the whole log-stream up to t_4 satisfies SI. When transaction t_5 arrives, however, an SI-violation is detected: t_5 is preceded by t_3 in so, which performs $\mathrm{W}(y, 1)$. Since t_5 cannot observe an earlier state, it must read 1 on y; however, the database served a stale write, leading to the event $\mathrm{R}(y, 0)$, thereby breaking its SI guarantee. *How efficiently can we implement this reasoning?*

The key idea of our monitoring algorithm is, upon receiving a transaction t, to walk backwards in co until it finds a matching snapshot—if not, a violation is reported. Walking is cheap: each step undoes the writes of one transaction, so the walk costs time proportional to the writes in $[t_{\text{MAX}}(t), t)$. A single interval can span the whole log, risking an $O(n^2)$ bound across the log-stream.

The crux of our efficiency guarantee is that, in aggregate, these intervals overlap very little: along any chain of the session and write orders, they are pairwise disjoint, each beginning only after its predecessor ends. Formally, the log decomposes into α chains, where α bounds the number of pairwise-concurrent transactions (and never exceeds the number of sessions), so our monitor runs in $O(n \cdot \alpha)$ time.

Despite its simplicity, we prove that this algorithm is essentially optimal under standard complexity-theoretic assumptions. In practice, the backwards traversal of our algorithm typically halts early, avoiding the factor α; in fact, we prove that on serializable log-streams the monitor runs in $O(n)$ time (which is also, trivially, optimal).

III. Background

In this section we develop background terminology, introduce Snapshot Isolation, and define gray-box monitoring.

General notation. For a relation R, we denote the reflexive closure, transitive closure, and inverse of R as $R^?$, R^+, and R^{-1}, respectively. We occasionally call $\langle a, b\rangle \in R$ an edge, and write it as $a \xrightarrow{R} b$. The composition of R_1 and R_2 is denoted by $R_1 ; R_2$. We write $\mathsf{irr}(R)$ to denote that a relation is irreflexive. Given a strict partial order R over a set S, a *chain* (resp. *antichain*) of R is a set of pairwise comparable (resp. incomparable) elements of S, and the *width* of R is the size of its largest antichain.

A. Database Logs

We set up basic notation on operations, transactions, and database logs.

Operations and transactions. We consider transactional key-value databases over a set of keys $\mathsf{Key} = \{x, y, \ldots\}$, a set of values Val, accessed by clients organized in sessions drawn from a set $\mathsf{Sess} = \{s_1, s_2, \ldots\}$. Clients interact with the database through *operations*: an operation is either a read $\mathrm{R}(x, v)$ or a write $\mathrm{W}(x, v)$, where $x \in \mathsf{Key}$ and $v \in \mathsf{Val}$; we write $\mathsf{key}(o)$ and $\mathsf{val}(o)$ for the key and value of an operation o. We let Op be the domain of operations, and distinguish the sets of read operations R and write operations W. For a key $x \in \mathsf{Key}$, we write W_x (resp., R_x) for the set of writes to x (resp., reads of x).

A *transaction* $t = \langle s, O, \mathsf{po}\rangle$ is composed of a session identifier $s \in \mathsf{Sess}$, a set of operations $O \subseteq \mathsf{Op}$, and a *program order* po, which is a strict total order over O. We denote by $\mathsf{sess}(t)$ the session of a transaction t. We often identify a transaction with its set of operations, writing, e.g., $t \cap \mathrm{W}_x$ for the writes of t on key x; we say that t *writes to* x if $t \cap \mathrm{W}_x \neq \emptyset$.

As is standard [6], [12], we assume, without loss of generality, that each transaction contains at most one write per key, and that no read of a key x is po-preceded by a write to x in the same transaction. When multiple writes to the same key are made in some transaction, only the po-maximal is visible to other transactions, and a read po-preceded by a write to the same key must return the value of the po-maximal such write. Both properties can be checked easily on the fly by a monitor. Note that, under these assumptions, a read cannot observe a write in its own transaction.

So far, each operation accesses a single key. Our model can be extended to support predicate operations with minimal changes; we treat these formally, and explain how to lift our monitor to handle them, in Section IV-E.

Database logs. A database monitor is external to the database, observing only the interaction of the database with its

clients: the transactions, as well as the order in which they are committed. Importantly, database internals are opaque to the monitor. We use the term "log-stream" to emphasize the fact that transactions are streamed to the monitor as soon as they commit, and the monitor has to process them on the fly in this order (i.e., it has to decide the consistency of a transaction as soon as the transaction commits).

Definition 1 (Log-stream)**.** *A* log-stream *is a sequence of transactions* $L = t_\bot, t_1, \ldots, t_m$*, where each* $t_i = \langle s_i, O_i, \mathsf{po}_i \rangle$*. We use* $\mathsf{txn}(L)$ *to refer to the set of transactions of* L*. The log-stream* L *induces three relations.*

- *The* commit order co *is the strict total order in which the transactions are streamed: for* $1 \le i, j \le m$*,* $t_i \xrightarrow{\mathsf{co}} t_j$ *iff* $i < j$*.*
- *The* session order so *is the restriction of* co *to pairs of transactions on the same session.*
- *The* write-write order $\mathsf{ww} = \bigcup_{x \in \mathsf{Key}} \mathsf{ww}_x$*, where* ww_x *is the restriction of* co *to the transactions that write* x*, orders the writers of each key.*

Finally, the initial transaction $t_\bot$ *writes to all keys and belongs to no session (hence* $\mathsf{sess}(t_\bot)$ *is undefined); nevertheless, we let* $t_\bot \xrightarrow{\mathsf{co}} t_i$ *and* $t_\bot \xrightarrow{\mathsf{so}} t_i$ *for all* $1 \le i \le m$*. Note that* $\mathsf{so}, \mathsf{ww} \subseteq \mathsf{co}$*.*

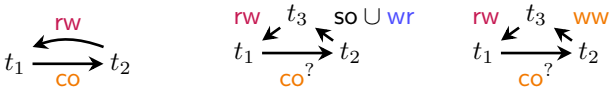
Parameters. We denote by n the *size of* L, measured as the number of its operations. We denote by m its number of transactions, by k its number of sessions, by ℓ its number of keys, by ν its number of distinct values, and by α the width of $(\mathsf{so} \cup \mathsf{ww})^+$; i.e., α bounds the number of pairwise-concurrent transactions, where two transactions are concurrent if neither precedes the other in $(\mathsf{so} \cup \mathsf{ww})^+$.

B. Isolation Levels

We follow the standard axiomatic specification of isolation levels based on a commit order co and a write-read relation wr [6], [12]. Whereas co is the order in which transactions were committed by the database, and is revealed in the log-stream, the write-read relation specifies the write that each read observes, and must be inferred.

Write-read relations. Given a log-stream L, a *write-read relation* for L is a relation $\mathsf{wr} \subseteq \bigcup_{x \in \mathsf{Key}} \mathrm{W}_x \times \mathrm{R}_x$, pairing each read to a write. For wr to be well-formed, it must be that (i) for each $w \xrightarrow{\mathsf{wr}} r$ we have $\mathsf{val}(w) = \mathsf{val}(r)$ and w and r belong to different transactions, (ii) wr^{-1} is a total function over reads, and (iii) $\mathsf{wr} \subseteq \mathsf{co}$. We call a well-formed write-read relation for L a *witness*. We occasionally view wr as a relation on transactions, i.e., $t_1 \xrightarrow{\mathsf{wr}} t_2$ iff $w \xrightarrow{\mathsf{wr}} r$ for some $w \in t_1$ and $r \in t_2$, and project it onto a key x by writing wr_x for $\mathsf{wr} \cap (\mathrm{W}_x \times \mathrm{R}_x)$.

A write-read relation wr further induces the *read-write* relation $\mathsf{rw} = \bigcup_{x \in \mathsf{Key}} \mathsf{rw}_x$, where $\mathsf{rw}_x = \mathsf{wr}_x^{-1}; \mathsf{ww}_x$, relating each read to the writes on the same key that are co-later than the write it observes. Note that ww depends only on co (Definition 1),



(a) SER (b) PREFIX (c) CONFLICT

Fig. 2: The forbidden patterns of SER and SI.

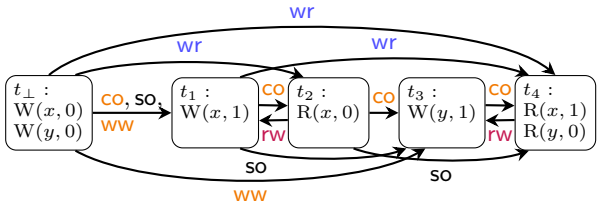


Fig. 3: The running example log-stream L restricted to $t_\bot, \ldots, t_4$, with a wr and the derived relations ww and rw.

whereas rw depends on wr and ww. In words, $t_1 \xrightarrow{\mathsf{rw}_x} t_2$ holds iff t_1 reads x from some transaction that is ww_x-before t_2; the read of t_1 is then *stale* with respect to t_2.

Isolation levels. We are now in position to define the SER and SI isolation levels, which are also visually depicted in Fig. 2. Each level constrains how stale reads may be, by forbidding cycles of a specific shape through rw.

Definition 2 (Serializability)**.** *A log-stream* L *satisfies* Serializability (SER) *if there exists a witness* wr *such that*

$$\mathsf{irr}(\mathsf{co} \,;\, \mathsf{rw}). \qquad \text{(SER)}$$

Intuitively, SER requires that transactions execute atomically in commit order, without any interleaving between them.

Definition 3 (Snapshot Isolation)**.** *A log-stream* L *satisfies* Snapshot Isolation (SI) *if there exists a witness* wr *such that*

$$\mathsf{irr}(\mathsf{co}^? \,;\, (\mathsf{so} \cup \mathsf{wr}) \,;\, \mathsf{rw}) \qquad \text{(PREFIX)}$$
$$\mathsf{irr}(\mathsf{co}^? \,;\, \mathsf{ww} \,;\, \mathsf{rw}). \qquad \text{(CONFLICT)}$$

In words, (PREFIX) states that if t' is an so- or wr-predecessor of a transaction t, then t must observe every write that committed no later than t'; that is, the reads of t cannot be stale with respect to such writes. The axiom (CONFLICT) states that two transactions writing a common key may not be concurrent, i.e., the co-earlier one must be visible to the co-later one.

Example. Figure 3 illustrates the definitions of SER and SI on a truncated version L' of the log-stream of Fig. 1. Note that the witness wr shown in the figure violates (SER), due to the cycle $t_1 \xrightarrow{\mathsf{co}} t_2 \xrightarrow{\mathsf{rw}} t_1$. In fact, every witness for L' violates (SER), since $t_\bot$ is the only transaction that can provide a valid value for $\mathrm{R}(x, 0)$ of t_2. Thus L' does not satisfy SER. On the other hand, the wr shown in the figure does not violate (PREFIX) or (CONFLICT), thus L' satisfies SI.

Gray-box monitoring. Finally, we state formally the gray-box monitoring problem for an arbitrary isolation level $\mathcal{I}$. Given a log-stream $L = t_\perp, t_1, \ldots, t_m$, decide, upon receiving each t_i whether the prefix $t_\perp, t_1, \ldots, t_i$ satisfies $\mathcal{I}$. In this work, we focus on $\mathcal{I}$ being SI.

IV. Monitoring for Snapshot Isolation

In this section we solve the monitoring problem for SI. We start by defining snapshots and establishing a key condition for when a log-stream satisfies SI in terms of the snapshots taken by its transactions (Section IV-A). On the way, we obtain a similar but simpler condition for SER, which helps keep our exposition intuitive. We then turn this condition into a monitoring algorithm for SI running in $O(n \cdot \alpha)$ time (Section IV-B), and further prove that, under the Hitting Set hypothesis, polynomial improvements over this bound are impossible (Section IV-C). Next, we discuss memory optimizations that are important for monitoring, and prove an unconditional lower bound on the memory required to monitor for SI in general (Section IV-D). Finally, we discuss how our algorithm can be lifted to handle predicate operations (Section IV-E).

Due to space limits, all proofs are delegated to Appendix A.

A. Snapshot-based Consistency Criteria

We begin with associating each transaction t of the log-stream L with the database state after t committed.

Definition 4 (Database state)**.** *Given a log-stream L, the* database state *at a transaction t' is the map $S_{t'} : \mathsf{Key} \to \mathsf{Val}$ assigning to each key the value of the* co*-latest write on it up to and including t'. Since $t_\perp$ initializes all keys, $S_{t'}$ is total for every t'.*

For a transaction $t \neq t_\perp$, we write $\mathsf{pred}(t)$ for the immediate co-predecessor of t, so that $S_{\mathsf{pred}(t)}$ is the database state just before t commits. We say that a transaction t can take a snapshot at an earlier time if the database state at that time contains the values that t reads.

Definition 5 (Snapshot)**.** *A transaction t can* take a snapshot at *a transaction t' if $t' \xrightarrow{\mathsf{co}} t$ and for all $\mathrm{R}(x, v) \in t$ we have $S_{t'}(x) = v$.*

Snapshots for SER. Under SER, a transaction must take a snapshot at its immediate predecessor, as stated in the following (straightforward) lemma.

Lemma 1. *A log-stream L satisfies* SER *iff every transaction $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$ can take a snapshot at $\mathsf{pred}(t)$.*

Lemma 1 immediately yields a monitor for SER, by maintaining the current database state on the fly. Upon processing transaction t, the monitor first checks whether every read of t matches the database state, and then updates it according to the writes of t. This processes each transaction in time proportional to its size, and thus $O(n)$ time in total, while the total space is $O(\ell)$.

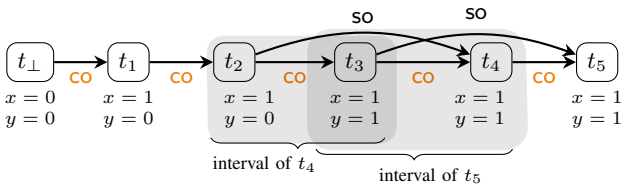


Fig. 4: On the example of Fig. 1, the interval of t_4 is $\{t_2, t_3\}$: its so predecessor t_2 is co-later than any of its (nonexistent) write-write obligations; indeed, t_4 can take a snapshot at t_2, shown below the transaction, whereas t_5, whose interval is $\{t_3, t_4\}$, can take no valid snapshot, and is rejected.

Snapshots for SI. SI relaxes the requirement that t must take its snapshot at $\mathsf{pred}(t)$. Instead, we characterize the set of permissible candidate snapshots by the following lemma.

Lemma 2. *A log-stream L satisfies* SI *iff every transaction $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$ admits a transaction t' such that (i) t can take a snapshot at t', and (ii) $t'' \xrightarrow{\mathsf{co}}{}^{?} t'$ for all t'' with $t'' \xrightarrow{\mathsf{so} \cup \mathsf{ww}} t$.*

Item (ii) confines the search for the snapshot taken by a transaction to an interval of the commit order. For $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$, let $t_{\mathrm{MAX}}(t)$ denote the co-maximal element of $\{\, t'' \mid t'' \xrightarrow{\mathsf{so} \cup \mathsf{ww}} t \,\}$, which is nonempty as it contains $t_\perp$. Thus, L satisfies SI iff every $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$ can take a snapshot at some t' with $t_{\mathrm{MAX}}(t) \xrightarrow{\mathsf{co}}{}^{?} t' \xrightarrow{\mathsf{co}} t$; we call such a snapshot *valid*. Note that the interval $[t_{\mathrm{MAX}}(t), t)$ is never empty, since it contains $t_{\mathrm{MAX}}(t)$. Figure 4 illustrates Lemma 2 on the running example.

B. Monitoring SI in $O(n \cdot \alpha)$ Time

We now develop Algorithm 1, which solves monitoring for SI with time complexity as stated in the following theorem.

Theorem 1. *Given a database log-stream L, monitoring for* SI *can be performed in $O(n \cdot \alpha)$ total time, where n is the number of operations and α is the width of $(\mathsf{so} \cup \mathsf{ww})^+$ (thus $\alpha \leq k$).*

In order to use Lemma 2, the monitor must determine, upon each transaction, whether it can take a valid snapshot. Upon processing a transaction t, the algorithm uses the store Δ (line 3) to record each update of t. Formally, for a write w on key x of a transaction t, it sets $\Delta[w] = S_{\mathsf{pred}(t)}(x)$.

To check whether a transaction t can take a valid snapshot, the algorithm first checks the reads of t against the current database state *currSnap* (line 11) and records in the set *wrongKeys* the set of keys of t whose value differs from *currSnap*; it then walks through the candidate transactions t' that t may take a valid snapshot at, in reverse co-order, stopping when $\mathit{wrongKeys} = \emptyset$ (line 14) or when it reaches $t_{\mathrm{MAX}}(t)$ (line 15).

Stepping from a candidate t' to its predecessor (line 21) amounts to undoing the writes of t'; to do so, for each

Algorithm 1 A monitor for SI

```
1:  procedure ApplyWrites(t)
2:      for w = W(x, v) ∈ t ∩ W do
3:          Δ[w] ← currSnap[x]; currSnap[x] ← v
4:  procedure MonitorSI(L)
5:      currSnap ← λx. ⊥;   Δ ← λw. ⊥
6:      lastTxn ← λs. t⊥;   lastWr ← λx. t⊥
7:      for each transaction t streamed by L do
8:          if t ≠ t⊥ then
9:              t_MAX(t) ← co-latest of
                    {lastTxn[sess(t)]} ∪ {lastWr[x] | t writes x}
10:             rdVal ← λx. (v if R(x, v) ∈ t, ⊥ otherwise)
11:             wrongKeys ← {x | R(x, v) ∈ t ∧ v ≠ currSnap[x]}
12:             c ← pred(t); found ← FALSE
13:             while TRUE do
14:                 if wrongKeys = ∅ then found ← TRUE; break
15:                 if c = t_MAX(t) then break
16:                 for w = W(x, v) ∈ c ∩ W s.t. rdVal[x] ≠ ⊥ do
17:                     if rdVal[x] = Δ[w] then
18:                         wrongKeys ← wrongKeys \ {x}
19:                     else
20:                         wrongKeys ← wrongKeys ∪ {x}
21:                 c ← pred(c)
22:             if found = FALSE then report violation
23:         ApplyWrites(t)
24:         if t ≠ t⊥ then lastTxn[sess(t)] ← t
25:         lastWr[x] ← t for each x written by t
```

$w = \mathrm{W}(x, v) \in t'$ the value of x is replaced with $\Delta[w]$. The set *wrongKeys* is updated as candidates are iterated (line 16) such that it contains the keys on which the reads of t disagree with the candidate snapshot, which takes time proportional to the size of each t'.

If there is a candidate t' such that the database state at t' agrees with all reads of t, then t can take a valid snapshot at t', and the search ends (line 14). If there is no such t', i.e., the walk reaches $t_{\mathrm{MAX}}(t)$ before finding a suitable candidate, then t can take no valid snapshot, and the monitor reports a violation of SI (line 22).

To locate $t_{\mathrm{MAX}}(t)$ (line 9), the algorithm keeps the last processed transaction $lastTxn[s]$ of each session s and the last writer $lastWr[x]$ of each key x (line 24): the direct so- and ww_x-predecessors of t that are co-maximal are exactly $lastTxn[\mathsf{sess}(t)]$ and $lastWr[x]$ for the keys x written by t, and $t_{\mathrm{MAX}}(t)$ is the co-latest among them.

Example. Consider again the running example of Fig. 1. The walk for t_4 starts at candidate t_3, finds $wrongKeys = \{y\}$, replaces the write of t_3 $\mathrm{W}(y, 1)$ with its delta 0, restoring $y = 0$, and accepts at candidate t_2 (line 14). This is illustrated in Fig. 5. The walk for t_5 starts at t_4, finds $wrongKeys = \{y\}$; since t_4 has no writes, the walk reaches $t_{\mathrm{MAX}}(t_5) = t_3$ without finding a matching snapshot (line 15), and the monitor reports a violation (line 22).

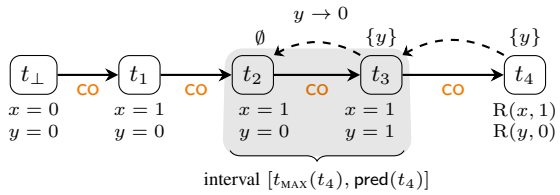


Fig. 5: The interval walk of Algorithm 1 for t_4 of the running example (Fig. 1). The value of *wrongKeys* at each step of the walk is shown above each transaction.

Time complexity. Upon processing transaction t, recording the deltas of its writes (line 23) costs $O(1)$ per write. The backwards walk for t visits the candidates in $\mathsf{cand}(t) = \{\, t' \mid t_{\mathrm{MAX}}(t) \xrightarrow{\mathsf{co}}{}^{?} t' \xrightarrow{\mathsf{co}} t \,\}$, and each candidate t' costs $O(1 + |t' \cap \mathrm{W}|)$ time: $O(1)$ for the visit itself, plus $O(1)$ per write of t' to undo it via its delta (line 16). All remaining work is $O(1)$ per operation of t, leading to the total running time

$$O\Big(n + \sum_{t} \sum_{t' \in \mathsf{cand}(t)} (1 + |t' \cap \mathrm{W}|)\Big).$$

The crux is that the intervals $\mathsf{cand}(t)$ overlap little; in particular, letting $C \subseteq \mathsf{txn}(L)$ be any chain of $(\mathsf{so} \cup \mathsf{ww})^+$, the intervals $\mathsf{cand}(t)$ for $t \in C$ are pairwise disjoint. Let $C_1, \ldots, C_\alpha$ be a decomposition of $\mathsf{txn}(L)$ into α chains; the total running time is then bounded by

$$O\Big(n + \sum_{i=1}^{\alpha} \sum_{t \in C_i} \sum_{t' \in \mathsf{cand}(t)} (1 + |t' \cap \mathrm{W}|)\Big)$$
$$\leq O\Big(n + \sum_{i=1}^{\alpha} \sum_{t' \in \mathsf{txn}(L)} (1 + |t' \cap \mathrm{W}|)\Big) = O(n \cdot \alpha).$$

Recovering the linear runtime for SER. Recall that, if L satisfies SER, Lemma 1 states that every transaction t can take a snapshot at $\mathsf{pred}(t)$. This makes the backward walk for t in Algorithm 1 stop immediately at $\mathsf{pred}(t)$ (line 14), leading to the following guarantee.

Lemma 3. *Algorithm 1 runs in $O(n)$ time on log-streams satisfying* SER.

More generally, Algorithm 1 runs faster the more recent the snapshots that transactions take: if every transaction takes a snapshot at one of the d newest candidates in its interval, then each walk visits at most d candidates, and the algorithm runs in $O(n \cdot d)$ time. The SER case yields $d = 1$, and d is typically small in practice.

C. A Lower Bound for Monitoring SI

We now show that the $O(n \cdot \alpha)$ running time of Theorem 1 is essentially optimal, conditional on the Hitting Set (HS) hypothesis [20]. The HS problem takes as input two sets $A, B \subseteq \{0, 1\}^d$ of Boolean vectors, and asks whether some $a \in A$ *hits* every $b \in B$, i.e., $\sum_i a[i] \cdot b[i] \neq 0$ for all $b \in B$. Letting $|A| = q^{\lambda_a}$ and $|B| = q^{\lambda_b}$ for constants $0 \leq \lambda_a, \lambda_b \leq 1$,

the (unbalanced) HS hypothesis [21] asserts that, for no $\epsilon > 0$, is there an algorithm for HS in $O(q^{\lambda_a+\lambda_b-\epsilon} \cdot \text{poly}(d))$ time. We prove the following theorem.

Theorem 2. *For any $0 \leq \lambda \leq 1$, every algorithm (monitoring or not) for deciding the* SI*-consistency of a log-stream of n operations, m transactions, and $(\mathsf{so} \cup \mathsf{ww})^+$-width $\alpha = \Theta(m^\lambda)$ must take $\Omega(n^{1+\lambda-\epsilon})$ time, for all $\epsilon > 0$, under the HS hypothesis.*

Since monitoring for SER can be performed in $O(n)$ time (recall our discussion after Lemma 1), Theorem 2 formally shows that monitoring for SI is a fundamentally more difficult problem. Moreover, this lower bound matches (wrt polynomial improvements) the upper bound of $O(n \cdot \alpha) = O(n^{1+\lambda})$ from Theorem 1, thus Algorithm 1 is essentially optimal.

D. Memory Requirements of Monitoring for SI

We now characterize the memory needed for monitoring SI.

The memory usage of Algorithm 1. Algorithm 1 uses $O(n)$ space, where n is the number of operations. The delta store Δ dominates the space usage, but it cannot be discarded in general: a later transaction with no writes may appear in a fresh session, in which case $t_{\text{MAX}}(t) = t_\perp$ and its walk may reach the oldest transactions. In practice, the memory usage of Algorithm 1 can be significantly improved to essentially $O(\ell)$, by employing a heuristic that soundly drops the old database states that are guaranteed to not provide valid snapshots for any future transaction t, by utilizing Lemma 2. Here we describe how to achieve this reduction. The key observation is that, since $t_{\text{MAX}}(t)$ is co-after or equal to the latest session predecessor t' of t, the snapshot taken by t cannot be co-before t'. Thus, it is safe to discard a transaction if all sessions have executed a transaction after it.

A practical obstacle to implementing this optimization is that a new session can start in the future, and SI allows its first transaction to read, in theory, from any past transaction. This prevents the monitor from using the above condition to discard old transactions. To solve this issue, we introduce two new special kinds of operations: a *session begin* operation $\text{B}(s)$ and a *session end* operation $\text{E}(s)$. We then require the first transaction $t_{\text{B}}(s)$ (the session begin transaction) in a session s to consist of the single operation $\text{B}(s)$ and the last $t_{\text{E}}(s)$ (the session end transaction) to consist of only $\text{E}(s)$. In a log-stream L with such transactions, we say that a session s is *open* at a transaction t, if $t_{\text{B}}(s) \xrightarrow{\mathsf{co}}{}^? t$ and $t_{\text{E}}(s) \not\xrightarrow{\mathsf{co}}{}^? t$; in particular, s is open at $t_{\text{B}}(s)$ but not at $t_{\text{E}}(s)$.

With this in hand, we can update the discarding criterion: a transaction t is safe to discard if all *sessions open at t* have executed a transaction after t. This is because, for any future session s, each of its transactions must take a snapshot after the begin transaction $t_{\text{B}}(s)$.

Incorporating this heuristic into Algorithm 1 is straightforward; we refer to Appendix A-D for pseudocode.

A practically grounded approach. In practice, even $O(\ell)$ memory could be prohibitive, when real-world workloads continuously add more keys. In such a case, our algorithm can be applied to a subset of keys of manageable size, such as a specific set of tables with heavy contention, while retaining its no-false-positive guarantees: any violation reported is a real violation, irrespectively of the non-monitored subset.

A space lower bound. Besides our practical heuristic in Algorithm 2, is it theoretically possible to monitor for SI using considerably less memory in the worst case? Although the linear dependency on n can indeed be dropped (we elaborate on this in the next paragraph), it turns out that no monitor can use asymptotically less than ν^ℓ space in the worst case.[1]

Theorem 3. *There are families of log-streams L, over $k = 2$ sessions, on which any monitor for* SI *must use $\Omega(\nu^\ell)$ space, for ℓ and ν respective keys and values in L.*

Since monitoring for SER takes only $O(\ell)$ space (recall our discussion after Lemma 1), Theorem 3 formally shows that monitoring for SI is fundamentally more memory-demanding. Intuitively, ν^ℓ counts the number of functions from keys to values, i.e., the number of all possible distinct database states. We prove that, in the worst case, a monitor has to remember all distinct states seen so far, because any of these can serve as the snapshot of a future transaction.

An essentially matching upper bound. Is there a monitor that uses provably sublinear memory? Indeed, monitoring for SI can be solved using $O(\ell \cdot \nu^\ell + k)$ memory and $O(n \cdot \alpha + m \cdot \ell)$ total time. Due to the relatively large factor of ν^ℓ in the memory, this monitor is of mostly theoretical interest; its significance is that its memory usage (essentially) matches the lower bound of Theorem 3. We refer to Appendix A-D for the pseudocode.

E. Predicates

So far, we have been working in a setting where each operation accesses a single key. This is not the case for *predicate operations*, which select the keys they access by a condition on the stored values, as in the `WHERE`-clause of a SQL statement. We show how to extend our model to handle such operations, and outline how Algorithm 1 changes in this setting.

Predicate reads. A *predicate read* operation is $\text{R}^*(P, M)$, where $P\colon \mathsf{Key} \times \mathsf{Val} \to \{\mathsf{T}, \mathsf{F}\}$ is a predicate on key-value pairs, and $M \subseteq \mathsf{Key}$ records which keys matched. It evaluates P on every key-value pair of the database state it observes, and returns the set M of keys on which P evaluates to T. For example, "`SELECT k FROM tab WHERE v > 0`" is a predicate read whose predicate is $P(x, v) = \mathsf{T}$ iff $v > 0$. When the statement also returns the values of the matched keys, e.g., "`SELECT k, v FROM tab WHERE v > 0`",

[1] This does not contradict the $O(n)$ memory complexity of Algorithm 1, since $n = \Omega(\nu^\ell)$ for the hard instances where the lower bound holds.

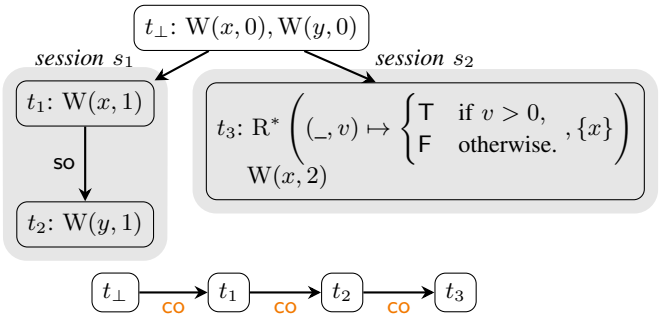


Fig. 6: A log-stream L in which t_3 contains a predicate read resulting from "`UPDATE tab SET v = 2 WHERE v > 0`": the first argument is a predicate P that returns T for every key with value greater than zero, and the second argument is the set $M = \{x\}$ of matched keys. Since x matches P, the predicate read is followed by a write to x. L is SI-consistent, as t_3 can take a snapshot at t_1, where $P(x,1) = \mathsf{T}$ and $P(y,0) = \mathsf{F}$, and the other transactions are write-only; in contrast, a snapshot at $t_\perp$ or t_2 is invalid, as $P(x,0) = \mathsf{F}$ and $P(y,1) = \mathsf{T}$, respectively.

these appear in the log as standard reads $\mathrm{R}(x,v)$, one for each $x \in M$, alongside the predicate read.

The monitor observes P and M, as those appear in the transaction, but not how the database evaluated P. To check a predicate read, the monitor evaluates P itself on each key for each database state that may have been observed.

Predicate writes. Predicate *writes* do not need individual treatment: a client statement that updates every key matching a condition appears in the log as a predicate read followed by ordinary writes on the matched keys; note that this means that one client statement may become several operations.

The write-read relation. A standard read $\mathrm{R}(x,v)$ is wr-paired with a write whose value is exactly v. A predicate read constrains its writers only through P and M: for each key x, the operation is wr-paired with a write w on x such that $P(x, \mathsf{val}(w)) = \mathsf{T}$ when $x \in M$, and $P(x, \mathsf{val}(w)) = \mathsf{F}$ when $x \notin M$. Intuitively, this treats the predicate read as a read on every key, with the well-formedness conditions of wr (Section III-B) also holding per key. The derived relation rw and the definitions of the isolation levels (Definitions 2 and 3) remain unchanged.

The definition of a snapshot (Definition 5) is lifted analogously: a transaction t can *take a snapshot at* t' if, additionally, for each predicate read $\mathrm{R}^*(P,M) \in t$ and key x,

$$P\big(x, S_{t'}(x)\big) = \mathsf{T} \iff x \in M.$$

With this definition, Lemma 2, the interval $[t_{\mathrm{MAX}}(t), t)$, and the notion of a valid snapshot carry over unchanged. Figure 6 shows a predicate read checked against a candidate snapshot.

Adapting the monitor. A candidate snapshot must now satisfy two kinds of constraints on each key x: the value constraints enforced by standard reads, and the ones enforced by predicate reads. Say that a value u is *admissible* at key x for t if (i) $rdVal[x] = \perp$ or $rdVal[x] = u$, i.e., t has no standard read on x or reads exactly u, and (ii) $P(x,u) = \mathsf{T} \iff x \in M$ for every predicate read $\mathrm{R}^*(P,M) \in t$.

The only changes to Algorithm 1 are where reads are tested against values. First, the initial check against the current database state, line 11, becomes

$$wrongKeys \leftarrow \{x \mid currSnap[x] \text{ is not admissible at } x \text{ for } t\}.$$

Second, the walk must revisit every key that a read of t constrains: if t contains a predicate read, that is every key, so the iteration of line 16 ranges over all writes $w = \mathrm{W}(x,v) \in c \cap \mathrm{W}$, dropping the filter $rdVal[x] \neq \perp$. Third, in the same loop, the test $rdVal[x] = \Delta[w]$ becomes the admissibility of $\Delta[w]$ at x for t. The remainder of the algorithm is unchanged.

A predicate read constrains ℓ keys, and thus counts as ℓ operations toward the size n. Each admissibility test evaluates the predicate reads of t, and therefore costs $O(1+r)$, where r is the number of predicate reads of t. Hence the bound of Theorem 1 becomes $O(n \cdot \alpha \cdot (1 + r_{\mathrm{MAX}}))$, where r_{MAX} is the maximum number of predicate reads per transaction.

V. Implementation

In this section we present Komodo—our implementation of Algorithm 1 as well as the surrounding monitoring architecture.[2] At a high level, the architecture (see Fig. 7) involves three main components interacting over the network: the client, the database, and the monitor. The client issues queries to the database, which we capture by instrumentation, similar to previous work on black-box isolation testers [6], [7], [8], [9], [10], [11], [12], [13], [14]. We configure the database to enable *change data capture* (CDC), making the database stream its updates in commit order. These two streams—queries from the client and the CDC stream—are then combined at the monitor into a single log-stream, which is the input to the algorithm.

Instrumenting the client. We use BenchBase (formerly OLTP-Bench) [22] as a client, since it supports a variety of databases and realistic workloads. With instrumentation, we collect the session order of issued transactions and *read operations*, including values returned by the database; for write operations, we simply note that a write happened, because the values written by an update query are not visible to the client, in general. The collected transactions with the written values, are then sent to the monitor via a Redis stream [23], which contain the written values. We note that client-side instrumentation is chosen for convenience in prototyping; a production implementation may set up a proxy between the database and the client, thereby leaving the client untouched.

[2]The implementation is available at https://github.com/lassemoldrup/Komodo

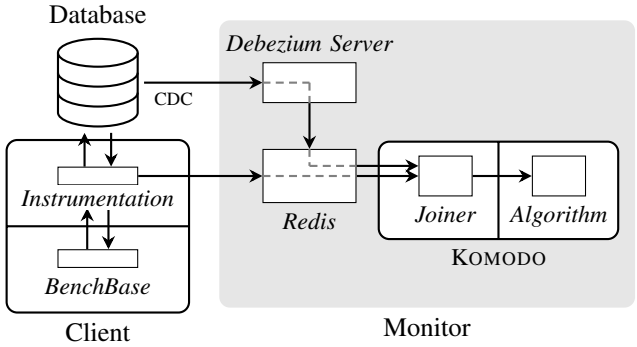


Fig. 7: The architecture of our implementation.

Using CDC to capture log-streams. Most modern databases support CDC in some form, and many specifically support Debezium [24], which provides a standardized format. We exploit this by connecting the database to Debezium Server [25], which can read the CDC stream and output a stream to our monitor via the Redis instance. Using this approach, our monitor only needs to support a single input format, allowing seamless integration of new databases.

This approach is quite general, but it relies on the CDC stream being in commit order, which is, currently, not guaranteed by all databases. As an example, the CDC stream of PostgreSQL [26], [27] is in the order that transactions are written to the write-ahead log, and this order is not guaranteed to match the order in which transactions become visible. However, the CDC order is also the order that replicas receive transactions in PostgreSQL [28], hence a client reading from multiple replicas arguably does not actually observe Snapshot Isolation.

Joining the client and CDC streams. In order to match transactions from the client and CDC streams, we create a special table where an id is written at the end of each transaction. This id then shows up in the CDC stream, allowing us to match the transactions reliably. Each transaction inserts a separate row to avoid contention on a shared row across concurrent sessions. The *Joiner* component of the monitor (Fig. 7) waits until both the client record and the complete CDC transaction are available, and combines the client-side reads with the writes from CDC.

A limitation of this approach is that the monitored workload contains no truly read-only transactions, which databases may treat differently. However, the id write is required for full monitoring: databases omit read-only transactions from their CDC streams (since they perform no changes), so such transactions could otherwise not be matched[3]. We emphasize that this caveat is purely technical; our Algorithm 1 handles read-only transactions correctly.

The algorithm. Finally, our algorithm processes the log-stream coming from the Joiner. It is implemented in C++ and closely follows the pseudocode of Algorithm 1 together with the memory optimization of Section IV-D.

[3]It appears that previous work [18] missed this case, leading to silently ignoring read-only transactions; as such, our architecture is more robust.

VI. EXPERIMENTS

In this section we perform an experimental evaluation of our monitor implementation. In Section VI-A, we perform experiments on the overhead of monitoring, while Section VI-B contains scalability experiments on the algorithm itself and a comparison to CHRONOS [18]. Finally, in Section VI-C, we report on bugs found by KOMODO stemming from faulty SI implementations. All experiments are performed on an Ubuntu 22.04 machine with a second-generation 2.3 GHz AMD Epyc CPU and 64 GB of memory.

Benchmarks. To simulate a realistic client workload, we use the following benchmarks from the standard BenchBase [22].

- TPC-C [29], a complex, popular benchmark.
- YCSB [30], slightly modified to support transactions; it allows fine-grained control over relevant parameters.
- SmallBank [31], a popular benchmark designed specifically to exercise non-serializable behaviors in SI databases.

Databases. We run our benchmarks on seven databases, namely MariaDB 12.3.2, MySQL 8.4.6, OceanBase 4.4.2, Oracle Database 23.26.2, Percona Server for MySQL 8.4.6-6, YugabyteDB 2025.2.2, and TiDB 8.4.0. These databases were chosen because their CDC implementations output transactions in commit order.[4] MariaDB, OceanBase, Oracle Database, YugabyteDB, and TiDB implement the specific version of SI that we target, while MySQL and Percona implement a slightly weaker version of SI (their implementations lack the first-committer-wins abort mechanism).

A. Experiments on Online Monitoring

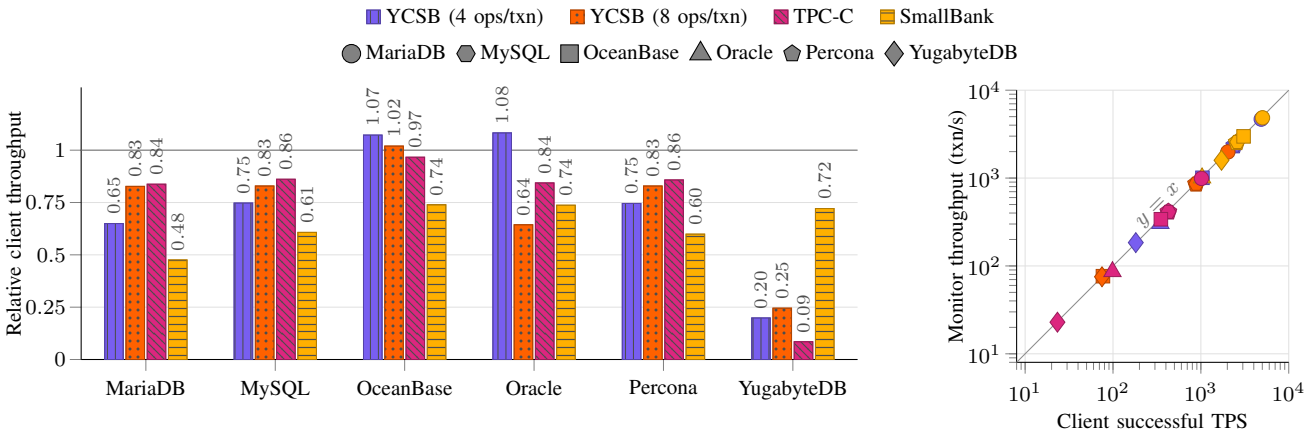
We are primarily interested in the following questions.

RQ1 What overhead does KOMODO incur on the database?
RQ2 What is the throughput of KOMODO?
RQ3 How much memory does KOMODO use in practice?
RQ4 Can KOMODO detect SI violations in real systems?

Setup. To answer these questions we run TPC-C and SmallBank at their default settings, which yields, 24 and 4 operations per transaction on average, respectively. Since YCSB lacks transactions originally, we create two variants by grouping every 4/8 operations into their own transaction. All benchmarks run on 10 sessions, and we run each benchmark on each database 10 times. Since the benchmarks contain randomness, we (randomly) fix 10 seeds per benchmark, reused in all databases. Each benchmark starts with a 10 s warm-up, followed by a measurement period of 120 s and 1000 s for the throughput and memory experiments, respectively.

Results on database overhead. To answer RQ1, we plot in Fig. 8a the ratio of the client throughput with vs without the

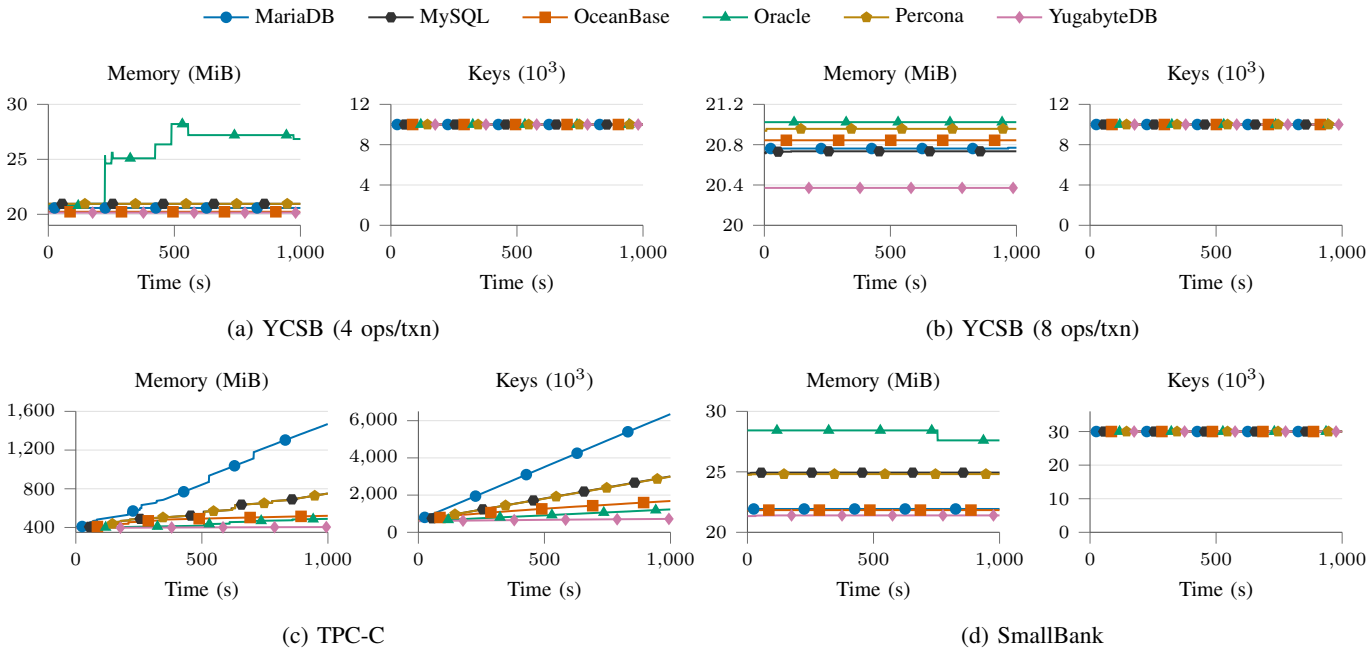
[4]Aside from TiDB, which we do not test in an online setting.



(a) Relative throughput on the client.

(b) Throughput comparison.

Fig. 8: Online monitoring performance.



(a) YCSB (4 ops/txn)

(b) YCSB (8 ops/txn)

(c) TPC-C

(d) SmallBank

Fig. 9: Monitor memory usage (left, MiB) and key counts (right, thousands) along benchmark execution.

monitor. This is sound because the bottleneck of the client in our setup is waiting for the database responses. We remark that KOMODO itself introduces no overhead, as it reads from a buffered stream; instead, overhead comes from CDC and the additional operations introduced.

The observed overhead is, generally, database- and benchmark-specific (some bars exceed 1; this is simply due to measurement fluctuations). YugabyteDB has a measurably higher overhead. At the time of writing, its CDC implementation is marked as "early access", which could explain the slowdown. Overall, the measurements indicate that monitoring a database via CDC can be relatively performant in practice, though the database may experience a reduction in throughput over specific loads.

Results on monitor throughput. To answer RQ2, we plot in Fig. 8b the throughput of the monitor against the throughput of the client. Most data points fall on the diagonal, with a few only slightly below it. This indicates that the monitor keeps pace with the rate at which the database commits transactions, across three orders of magnitude of throughput.

Results on monitor memory usage. To answer RQ3, we plot in Fig. 9 the memory usage during benchmark execution. Since our monitor (or any monitor in general) has a natural memory dependency on the number of database keys, we also plot the number of keys accessed over time. All benchmarks besides TPC-C utilize a fixed number of keys, and this results in constant memory usage across all databases, modulo occasional fluctuations in Oracle. At closer inspection, these fluctuations

are due to CDC occasionally flushing larger chunks of logged transactions. On the other hand, in TPC-C, the number of database keys increases over time, leading to a proportional increase in memory. The rate of increase varies per database, which is due to variance in database throughput. MariaDB delivers the highest throughput for TPC-C (see Fig. 8b), and thus utilizes more keys (and hence more memory) at a higher pace. We note that TPC-C deliberately creates contention on a fixed subset of keys, where anomalies are more likely to arise; to achieve constant memory in practice, KOMODO could be configured to monitor only those.

SI **violations.** Finally, we answer RQ4 in the affirmative; KOMODO reports violations in *all* experiments on MySQL and Percona, which implement a slightly weakened SI.

B. Offline Experiments

Here we focus on the performance of the KOMODO algorithm separately from the surrounding architecture, and investigate how it compares to the recent CHRONOS SI checker [18]. CHRONOS uses additional information in the form of transaction start timestamps extracted from the database to avoid searching for the snapshot of each transaction, hence we evaluate whether the lack of such information disadvantages KOMODO. We formulate the following questions.

RQ5 How does the performance of the KOMODO algorithm compare to the state-of-the-art CHRONOS SI checker?
RQ6 How many snapshots does KOMODO consider per transaction, on average, across different levels of contention?

Setup. We use the TiDB database, as in the experimental setup of CHRONOS [18], which we similarly modify to extract the start timestamps required by CHRONOS, and to output log files. For RQ5, we run both tools on logs generated by the TPC-C and YCSB benchmarks. CHRONOS does not assume its input transactions to follow the commit order, but instead has an initial sorting step, which we disable for fairness. To answer RQ6, we run YCSB on TiDB while scaling the number of sessions and the size of each transaction. YCSB uses a Zipfian distribution to generate keys, which has a parameter θ that determines the skew; higher values of θ mean that the same keys are chosen more often (more contention), while $\theta = 0$ creates a uniform distribution (less contention). We run our experiments at various values of θ to see how contention influence the result.

Results. Figure 10 shows the runtime comparison between KOMODO and CHRONOS, where each data point represents one log. All points lie below the diagonal, indicating that KOMODO is faster in all cases. This may be partially attributed to different implementation languages (C++ for KOMODO, Java for CHRONOS), but the results clearly indicate that KOMODO not using start timestamps does not make for a major disadvantage.

Figure 11 shows the average snapshots considered per transaction by KOMODO, as a function of the number of sessions

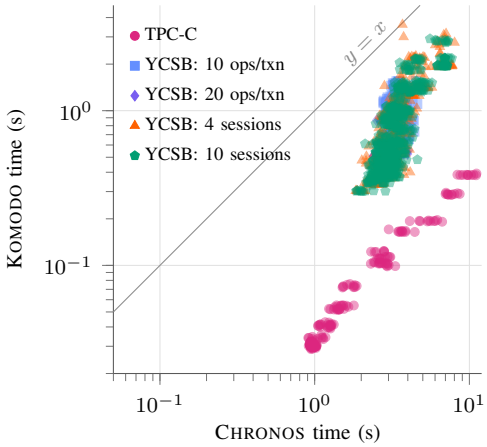


Fig. 10: CHRONOS vs KOMODO checking time.

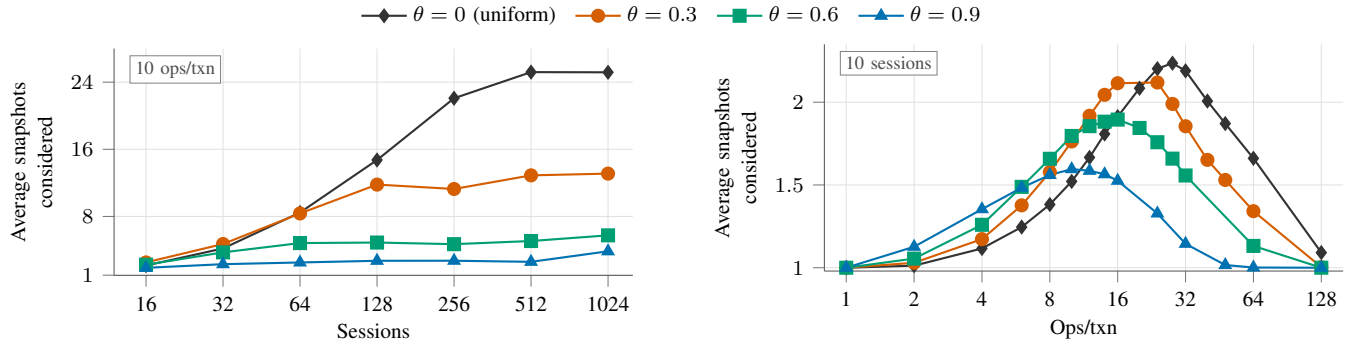
(Fig. 11a) and the transaction size (Fig. 11b). Scaling the number of sessions shows an expected increasing trend, since more sessions make the log non-serializable. Nevertheless, the scaling is small (approximately logarithmic—flattening out at high session counts) and the number of snapshots considered remains relatively small, even at unrealistically large session counts, explaining the practically linear-time performance of KOMODO. Interestingly, scaling the transaction size shows a maximal value that depends on the value of θ. This shape comes from two opposing effects of larger transaction size: (i) more transactions can be committed between the start and end of a transaction (increasing snapshots considered), and (ii) more transactions become ww-ordered, allowing KOMODO to stop traversing sooner (decreasing snapshots considered).

C. Bugs found

KOMODO has detected SI anomalies in two popular databases, OceanBase and TiDB, revealing a buggy implementation of SI.

OceanBase. We used an LLM to identify suspicious code and then utilized simple fuzzing to exercise this code with KOMODO running as a monitor. After two hours, KOMODO detected an SI violation caused by sampled scans (`SELECT ... SAMPLE`): once a major compaction has run (merging SSTables—files on disk that store key-value pairs), the sampled scan reads the latest compacted data instead of the snapshot data. For example, within one transaction, a plain read returns $\mathrm{R}(x, 10)$, while a sampled read returns $\mathrm{R}(x, 11)$, i.e., values from two separate snapshots.

TiDB. In the offline experiments, KOMODO rejected one YCSB log (256 sessions, $\theta = 0.3$): a read-only transaction observed, on four keys, values written by transactions that commit after its snapshot is taken. We traced this to Badger, the storage engine of TiDB's unistore backend. To answer point reads (reads of a specific key), a hash index is used to locate the key in an SSTable. In case of a hash collision between two keys in different blocks, the read will fall back to a search that erroneously ignores the start timestamp.



(a) Sessions vs snapshots considered.

(b) Transaction size vs snapshots considered.

Fig. 11: Results on offline experiments.

VII. Related Work

Here we discuss related work, starting with a comparison to Emme/Enne and Chronos/Aion, since these come the closest to our setting. Then, we discuss broader related work.

Emme/Enne. Clark et al. [16], [17] develop two white-box checkers, Emme and Enne, that rely on transaction ordering information. Emme supports SER and SI by using an expected version order and an expected version set extracted using CDC and other database-dependent APIs. The expected version order defines the ordering of write operations to individual keys, whereas the expected version set contains, for each predicate operation, the snapshot taken by its transaction. Enne supports SER and makes the stronger assumption about the extracted information that it is given an expected serialization order. Komodo can be seen as an extension of Enne to SI. Compared to Emme's SI checker, Komodo has a number of advantages: (i) it is online; (ii) it relies only on CDC with Debezium for ordering information, yielding a stable gray-box API; (iii) it does not require an expected version set, thereby supporting more databases; and (iv) it does not require that each value written is unique.

Chronos/Aion. Li et al. [18] develop two white-box testers for SI: Chronos, which is offline, and Aion, which is online. In their setting, each transaction is assigned two timestamps: a start timestamp and an end timestamp. These are extracted as white-box information from timestamp-based databases using a variety of database-dependent techniques. The end timestamps define the commit order, and the start timestamps define the snapshot taken by each transaction. Unlike Komodo, Aion does not assume transactions are given in commit order, making the two approaches complementary. Komodo is considerably easier to apply to new databases, as it does not require modifying the database.

Other isolation monitors. Several other proposals have been made for online SER checking. Zellag and Kemme [32] propose a technique for monitoring for SER violations on systems that correctly implement a weaker isolation level. RushMon [33] is a low-overhead white-box monitor that estimates the number of SER violations in a system. Tan et al. [8] propose Cobra, an offline black-box tester for SER that uses an SMT solver; an online version is proposed by introducing fence transactions that split the workload into rounds that can be checked independently. Finally, Leopard [34], [35] is a black-box online bug-finding tool that can find bugs under various isolation levels in various databases, assuming knowledge of the isolation level implementation.

Black-box testing. A line of work has studied the efficiency of black-box isolation testing without a commit order, a problem which is NP-complete for SER [15]. Later, Biswas and Enea [6] categorize a number of other common isolation levels, including SI, as either polynomial-time testable or NP-complete. Since then, SMT solvers have been utilized for testing for SER [8], [14] and SI [9], [10], [14], whereas other work has developed polynomial-time algorithms for testing weak isolation [11], [13].

Fuzzing. Several works develop fuzzing/query-synthesis techniques for generating workloads that can reveal isolation anomalies, such as Elle [7], DynSQL [36], Troc [37], and SQLancer++ [38]. Our work is, naturally, orthogonal; Komodo is deployed on the database side and monitors the interaction generated by such workloads in real time.

Shared memory. The monitoring of transactional consistency has also been studied in the context of shared memory, where transactions mark segments of atomicity invariants [39], [40], [41]. The problem is often expressed as detecting violations of conflict-serializability, and its efficiency has been a subject of extensive work [42], [43], [44], [45], [46].

VIII. Conclusion

We have introduced Komodo, an optimal gray-box database monitor for Snapshot Isolation. We show that Komodo keeps pace with high live commit throughput across seven popular databases and finds bugs in the Snapshot Isolation implementation of two. Natural future work concerns the efficiency of monitoring for other isolation levels, such as Read Committed.

Acknowledgement

We have made use of AI (Claude Code and OpenAI Codex) to help implement KOMODO, set up databases, run experiments, and plot data.

References

[1] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," *SIGMOD Rec.*, vol. 24, no. 2, pp. 1–10, May 1995. DOI: 10.1145/568271.223785.

[2] PostgreSQL Global Development Group. "PostgreSQL," Accessed: Sep. 9, 2026. [Online]. Available: https://www.postgresql.org/.

[3] Oracle. "Oracle Database," Accessed: Sep. 9, 2026. [Online]. Available: https://www.oracle.com/database/.

[4] Microsoft. "Microsoft SQL Server," Accessed: Sep. 9, 2026. [Online]. Available: https://www.microsoft.com/en-us/sql-server/.

[5] Jepsen. "Analyses," Accessed: Aug. 10, 2026. [Online]. Available: https://jepsen.io/analyses.

[6] R. Biswas and C. Enea, "On the complexity of checking transactional consistency," vol. 3, no. OOPSLA, Oct. 2019, Art. no. 165. DOI: 10.1145/3360591.

[7] K. Kingsbury and P. Alvaro, "Elle: Inferring isolation anomalies from experimental observations," *Proc. VLDB Endow.*, vol. 14, no. 3, pp. 268–280, 2020. DOI: 10.14778/3430915.3430918.

[8] C. Tan, C. Zhao, S. Mu, and M. Walfish, "Cobra: Making transactional key-value stores verifiably serializable," in *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, USENIX Association, 2020, pp. 63–80.

[9] J. Zhang, Y. Ji, S. Mu, and C. Tan, "Viper: A Fast Snapshot Isolation Checker," in *Proceedings of the Eighteenth European Conference on Computer Systems*, ser. EuroSys '23, New York, NY, USA: Association for Computing Machinery, May 2023, pp. 654–671. DOI: 10.1145/3552326.3567492.

[10] K. Huang et al., "Efficient Black-Box Checking of Snapshot Isolation in Databases," *Proc. VLDB Endow.*, vol. 16, no. 6, pp. 1264–1276, Feb. 2023. DOI: 10.14778/3583140.3583145.

[11] S. Liu, L. Gu, H. Wei, and D. Basin, "Plume: Efficient and complete black-box checking of weak isolation levels," vol. 8, no. OOPSLA, Oct. 2024, Art. no. 302. DOI: 10.1145/3689742.

[12] A. Bouajjani, C. Enea, and E. Román-Calvo, "On the complexity of checking mixed isolation levels for SQL transactions," in *Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part IV*, R. Piskac and Z. Rakamaric, Eds., ser. Lecture Notes in Computer Science, vol. 15934, Springer, 2025, pp. 315–337. DOI: 10.1007/978-3-031-98685-7_15.

[13] L. Møldrup and A. Pavlogiannis, "AWDIT: an optimal weak database isolation tester," *Proc. ACM Program. Lang.*, vol. 9, no. PLDI, pp. 1540–1564, 2025. DOI: 10.1145/3742465.

[14] Z. Cai, S. Liu, H. Wei, Y. Chen, and A. Pan, "Fast Verification of Strong Database Isolation," *Proceedings of the VLDB Endowment*, vol. 19, no. 4, pp. 563–575, Dec. 2025. DOI: 10.14778/3785297.3785300.

[15] C. H. Papadimitriou, "The serializability of concurrent database updates," *J. ACM*, vol. 26, no. 4, pp. 631–653, Oct. 1979. DOI: 10.1145/322154.322158.

[16] J. Clark, A. F. Donaldson, J. Wickerson, and M. Rigger, "Validating database system isolation level implementations with version certificate recovery," in *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*, ACM, 2024, pp. 754–768. DOI: 10.1145/3627703.3650080.

[17] J. Clark, M. Rigger, J. Wickerson, and A. Donaldson, "Bringing order to practical validation of database isolation levels," *ACM Trans. Comput. Syst.*, vol. 44, no. 3, Apr. 2026, Art. no. 15. DOI: 10.1145/3797871.

[18] H. Li et al., "Online timestamp-based transactional isolation checking of database systems," in *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*, IEEE, 2025, pp. 3738–3750. DOI: 10.1109/ICDE65448.2025.00279.

[19] A. Cerone, G. Bernardi, and A. Gotsman, "A Framework for Transactional Consistency Models with Atomic Visibility," in *26th International Conference on Concurrency Theory (CONCUR 2015)*, L. Aceto and D. de Frutos Escrig, Eds., ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 42, Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2015, pp. 58–71. DOI: 10.4230/LIPIcs.CONCUR.2015.58.

[20] A. Abboud, V. V. Williams, and J. Wang, "Approximation and fixed parameter subquadratic algorithms for radius and diameter in sparse graphs," in *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, ser. SODA '16, Arlington, Virginia: Society for Industrial and Applied Mathematics, 2016, pp. 377–391.

[21] Y. Kirkpatrick and V. Vassilevska Williams, "Shortest paths in multimode graphs," *CoRR*, vol. abs/2506.22261, 2025. DOI: 10.48550/ARXIV.2506.22261. arXiv: 2506.22261.

[22] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudré-Mauroux, "Oltp-bench: An extensible testbed for benchmarking relational databases," *Proc. VLDB Endow.*, vol. 7, no. 4, pp. 277–288, 2013. DOI: 10.14778/2732240.2732246.

[23] Redis. "Redis Streams," Accessed: Sep. 9, 2026. [Online]. Available: https://redis.io/docs/latest/develop/data-types/streams/.

[24] Debezium Community. "Debezium 3.6 Documentation: Architecture," Accessed: Sep. 9, 2026. [Online]. Available: https://debezium.io/documentation/reference/stable/architecture.html.

[25] Debezium Community. "Debezium 3.6 Documentation: Debezium Server," Accessed: Sep. 9, 2026. [Online]. Available: https://debezium.io/documentation/reference/stable/operations/debezium-server.html.

[26] PostgreSQL Global Development Group. "PostgreSQL 18 Documentation: Logical Decoding Output Plugins," Accessed: Sep. 9, 2026. [Online]. Available: https://www.postgresql.org/docs/18/logicaldecoding-output-plugin.html.

[27] A. Aasma. "Master-slave visibility order." pgsql-hackers mailing list, August 28, Accessed: Sep. 9, 2026. [Online]. Available: https://www.postgresql.org/message-id/CA+CSw_uNoGCy8p17fxPhMUC+-TWspLEPYLhfJeTm1GdWgVDxRA@mail.gmail.com.

[28] A. Freund. "Re: Logical changeset generation v3 – git repository." pgsql-hackers mailing list, December 13, Accessed: Sep. 9, 2026. [Online]. Available: https://www.postgresql.org/message-id/20121213172859.GA7991@awork2.anarazel.de.

[29] Transaction Processing Performance Council. "Overview of the TPC-C benchmark." Revision 5.11.0, Accessed: Jun. 22, 2026. [Online]. Available: https://www.tpc.org/tpcc/default5.asp.

[30] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC 2010, Indianapolis, Indiana, USA, June 10-11, 2010*, J. M. Hellerstein, S. Chaudhuri, and M. Rosenblum, Eds., ACM, 2010, pp. 143–154. DOI: 10.1145/1807128.1807152.

[31] M. Alomari, M. J. Cahill, A. D. Fekete, and U. Röhm, "The cost of serializability on platforms that use snapshot isolation," in *Proceedings of the 24th International Conference on Data Engineering, ICDE 2008, April 7-12, 2008, Cancún, Mexico*, G. Alonso, J. A. Blakeley, and A. L. P. Chen, Eds., IEEE Computer Society, 2008, pp. 576–585. DOI: 10.1109/ICDE.2008.4497466.

[32] K. Zellag and B. Kemme, "Consistency anomalies in multi-tier architectures: Automatic detection and prevention," *VLDB J.*, vol. 23, no. 1, pp. 147–172, 2014. DOI: 10.1007/S00778-013-0318-X.

[33] Z. Shang, J. X. Yu, and A. J. Elmore, "Rushmon: Real-time isolation anomalies monitoring," in *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, G. Das, C. M. Jermaine, and P. A. Bernstein, Eds., ACM, 2018, pp. 647–662. DOI: 10.1145/3183713.3196932.

[34] K. Li et al., "Leopard: A black-box approach for efficiently verifying various isolation levels," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 722–735. DOI: 10.1109/ICDE55515.2023.00061.

[35] S. Weng et al., "A high-performance black-box bug-finding tool for database isolation levels," *IEEE Trans. Knowl. Data Eng.*, vol. 38, no. 6, pp. 3349–3364, 2026. DOI: 10.1109/TKDE.2026.3676641.

[36] Z.-M. Jiang, J.-J. Bai, and Z. Su, "DynSQL: Stateful fuzzing for database management systems with complex and valid SQL query generation," in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA: USENIX Association, Aug. 2023, pp. 4949–4965.

[37] W. Dou et al., "Detecting isolation bugs via transaction oracle construction," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23, Melbourne, Victoria, Australia: IEEE Press, 2023, pp. 1123–1135. DOI: 10.1109/ICSE48619.2023.00101.

[38] S. Zhong and M. Rigger, "Scaling automated database system testing," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '26, USA: Association for Computing Machinery, 2026, pp. 1677–1692. DOI: 10.1145/3779212.3790215.

[39] S. Lu, J. Tucek, F. Qin, and Y. Zhou, "Avio: Detecting atomicity violations via access interleaving invariants," in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS XII, San Jose, California, USA: Association for Computing Machinery, 2006, pp. 37–48. DOI: 10.1145/1168857.1168864.

[40] M. Xu, R. Bodík, and M. D. Hill, "A serializability violation detector for shared-memory server programs," in *Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation, Chicago, IL, USA, June 12-15, 2005*, V. Sarkar and M. W. Hall, Eds., ACM, 2005, pp. 1–14. DOI: 10.1145/1065010.1065013.

[41] S. Park, S. Lu, and Y. Zhou, "Ctrigger: Exposing atomicity violation bugs from their hiding places," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS XIV, Washington, DC, USA: Association for Computing Machinery, 2009, pp. 25–36. DOI: 10.1145/1508244.1508249.

[42] C. Flanagan, S. N. Freund, and J. Yi, "Velodrome: A sound and complete dynamic atomicity checker for multithreaded programs," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '08, Tucson, AZ, USA: Association for Computing Machinery, 2008, pp. 293–303. DOI: 10.1145/1375581.1375618.

[43] S. Biswas, J. Huang, A. Sengupta, and M. D. Bond, "Doublechecker: Efficient sound and precise atomicity checking," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '14, Edinburgh, United

Kingdom: Association for Computing Machinery, 2014, pp. 28–39. DOI: 10.1145/2594291.2594323.

[44] U. Mathur and M. Viswanathan, "Atomicity checking in linear time using vector clocks," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20, Lausanne, Switzerland: Association for Computing Machinery, 2020, pp. 183–199. DOI: 10.1145/3373376.3378475.

[45] X. Ma et al., "Regiontrack: A trace-based sound and complete checker to debug transactional atomicity violations and non-serializable traces," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 1, Dec. 2021, Art. no. 7. DOI: 10.1145/3412377.

[46] H. C. Tunç, Y. Dong, and A. Pavlogiannis, "Fast atomicity monitoring," *Proc. ACM Program. Lang.*, vol. 10, no. PLDI, Jun. 2026, Art. no. 170. DOI: 10.1145/3808248.

[47] R. P. Dilworth, "A decomposition theorem for partially ordered sets," *Annals of Mathematics*, vol. 51, no. 1, pp. 161–166, 1950.

APPENDIX A
PROOFS OF SECTION IV

Proposition 1 (Persistence of violations)**.** *Let L be a log-stream and let L' be a prefix of L. If L satisfies* SI*, then so does L'; the same holds for* SER*.*

Proof. Let wr be a witness for L, and let wr' be its restriction to the operations of L'. Every read of L' is paired by wr with a write that is co-earlier, hence belongs to L', so wr' is a witness for L'. The relations $\mathsf{so}, \mathsf{ww}, \mathsf{rw}$ induced on L' are the restrictions of those induced on L, so every cycle forbidden by (SER), or by (PREFIX) and (CONFLICT), in L' is also one in L. □

As a consequence of Proposition 1, a monitor may report a violation upon the first transaction whose prefix is inconsistent, and every later prefix is inconsistent as well.

A. Proofs of Section IV-A

We prove the characterizations of SER and SI via snapshots.

Lemma 1. *A log-stream L satisfies* SER *iff every transaction $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$ can take a snapshot at $\mathsf{pred}(t)$.*

Proof. First, assume that L satisfies SER for some witness wr, and consider a read $\mathrm{R}(x, v)$ of a transaction t, paired by wr with a write of some transaction t_1. Since $\mathsf{wr} \subseteq \mathsf{co}$, we have $t_1 \xrightarrow{\mathsf{co}} t$. We argue that the write of t_1 on x is the co-latest write on x before t. Otherwise, some $t_2 \neq t$ writing x would satisfy $t_1 \xrightarrow{\mathsf{co}} t_2 \xrightarrow{\mathsf{co}} t$; then $t_1 \xrightarrow{\mathsf{ww}_x} t_2$, hence $t \xrightarrow{\mathsf{rw}_x} t_2$, and the cycle $t_2 \xrightarrow{\mathsf{co}} t \xrightarrow{\mathsf{rw}_x} t_2$ contradicts (SER). Since the transactions up to and including $\mathsf{pred}(t)$ are exactly those strictly co-before t, we conclude $v = S_{\mathsf{pred}(t)}(x)$, and thus t can take a snapshot at $\mathsf{pred}(t)$.

Conversely, assume that every transaction can take a snapshot at its immediate co-predecessor, and define wr by pairing each read $\mathrm{R}(x, v)$ of each transaction t with the co-latest write on x strictly before t. This write exists, since $t_\perp$ writes all keys; its value is $S_{\mathsf{pred}(t)}(x) = v$ by assumption; and it belongs to a transaction other than t. It is straightforward to check that this wr is a witness for L: wr^{-1} is a total function on reads, and $\mathsf{wr} \subseteq \mathsf{co}$. Finally, (SER) holds. Otherwise, there would be a cycle $t_2 \xrightarrow{\mathsf{co}} t_3 \xrightarrow{\mathsf{rw}_x} t_2$ for some key x, meaning that t_3 reads x from a writer t_1 with $t_1 \xrightarrow{\mathsf{ww}_x} t_2 \xrightarrow{\mathsf{co}} t_3$, contradicting that wr pairs the reads of t_3 with co-latest writers. □

Lemma 2. *A log-stream L satisfies* SI *iff every transaction $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$ admits a transaction t' such that (i) t can take a snapshot at t', and (ii) $t'' \xrightarrow{\mathsf{co}}{}^{?} t'$ for all t'' with $t'' \xrightarrow{\mathsf{so} \cup \mathsf{ww}} t$.*

Proof. Call a *snapshot assignment* a map snap assigning to each $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$ a transaction $\mathsf{snap}(t)$ satisfying Items (i) and (ii); the lemma states that L satisfies SI iff a snapshot assignment exists.

First, assume that there exists a witness wr, and fix $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$. Let t' be the co-maximal element of $\{\, t'' \mid t'' \xrightarrow{\mathsf{so} \cup \mathsf{wr} \cup \mathsf{ww}} t \,\}$; the set is nonempty, as $t_\perp \xrightarrow{\mathsf{so}} t$, and all its elements are co-before t, as $\mathsf{so}, \mathsf{wr}, \mathsf{ww} \subseteq \mathsf{co}$. Item (ii) holds by maximality, since $\mathsf{so} \cup \mathsf{ww} \subseteq \mathsf{so} \cup \mathsf{wr} \cup \mathsf{ww}$.

For Item (i), note that $t' \xrightarrow{\mathsf{co}} t$ and consider a read $\mathrm{R}(x, v)$ of t, paired by wr with a write of some transaction t_1; then $t_1 \xrightarrow{\mathsf{co}}{}^{?} t'$, and it suffices to show that t_1 is the co-latest writer of x up to t', as then $S_{t'}(x) = v$. Otherwise, some t_2 writing x would satisfy $t_1 \xrightarrow{\mathsf{co}} t_2 \xrightarrow{\mathsf{co}}{}^{?} t'$; then $t_1 \xrightarrow{\mathsf{ww}_x} t_2$, hence $t \xrightarrow{\mathsf{rw}_x} t_2$, and we exhibit a forbidden cycle by splitting on the edge $t' \xrightarrow{\mathsf{so} \cup \mathsf{wr} \cup \mathsf{ww}} t$ given by the construction of t'. If $t' \xrightarrow{\mathsf{so} \cup \mathsf{wr}} t$, the cycle $t_2 \xrightarrow{\mathsf{co}}{}^{?} t' \xrightarrow{\mathsf{so} \cup \mathsf{wr}} t \xrightarrow{\mathsf{rw}_x} t_2$ violates (PREFIX); if instead $t' \xrightarrow{\mathsf{ww}} t$, the cycle $t_2 \xrightarrow{\mathsf{co}}{}^{?} t' \xrightarrow{\mathsf{ww}} t \xrightarrow{\mathsf{rw}_x} t_2$ violates (CONFLICT).

Conversely, let snap be a snapshot assignment. We define a wr relation by pairing each read $\mathrm{R}(x, v)$ of each transaction t with the co-latest write on x up to $\mathsf{snap}(t)$: this write exists since $t_\perp$ writes all keys, its value is $S_{\mathsf{snap}(t)}(x) = v$ by Item (i) and Definition 5, and its transaction is co-before t, since $\mathsf{snap}(t) \xrightarrow{\mathsf{co}} t$.

Hence this wr is a witness for L, and it remains to verify (PREFIX) and (CONFLICT). Both verifications rest on the same observation: there are no t_2, t_3 and key x with $t_3 \xrightarrow{\mathsf{rw}_x} t_2$ and $t_2 \xrightarrow{\mathsf{co}}{}^{?} \mathsf{snap}(t_3)$. Indeed, unfolding rw_x, wr pairs the read of t_3 on x with the co-latest writer t_1 of x up to $\mathsf{snap}(t_3)$, and $t_1 \xrightarrow{\mathsf{ww}_x} t_2$; but then t_2 writes x, is co-after t_1, and is not co-after $\mathsf{snap}(t_3)$, contradicting the co-latestness of t_1.

Now, if (PREFIX) failed, there would be a cycle $t_2 \xrightarrow{\mathsf{co}}{}^{?} t_4 \xrightarrow{\mathsf{so} \cup \mathsf{wr}} t_3 \xrightarrow{\mathsf{rw}_x} t_2$ for some key x. If $t_4 \xrightarrow{\mathsf{so}} t_3$, then $t_4 \xrightarrow{\mathsf{co}}{}^{?} \mathsf{snap}(t_3)$ by Item (ii); if instead $t_4 \xrightarrow{\mathsf{wr}} t_3$, the canonical wr pairs reads of t_3 with writers up to $\mathsf{snap}(t_3)$, so again $t_4 \xrightarrow{\mathsf{co}}{}^{?} \mathsf{snap}(t_3)$. Either way $t_2 \xrightarrow{\mathsf{co}}{}^{?} t_4 \xrightarrow{\mathsf{co}}{}^{?} \mathsf{snap}(t_3)$, and the observation applies. Similarly, if (CONFLICT) failed, there would be a cycle $t_2 \xrightarrow{\mathsf{co}}{}^{?} t_4 \xrightarrow{\mathsf{ww}} t_3 \xrightarrow{\mathsf{rw}_x} t_2$ for some key x; since $\mathsf{ww} \subseteq \mathsf{so} \cup \mathsf{ww}$, Item (ii) gives $t_4 \xrightarrow{\mathsf{co}}{}^{?} \mathsf{snap}(t_3)$, so $t_2 \xrightarrow{\mathsf{co}}{}^{?} \mathsf{snap}(t_3)$, and the observation applies. □

B. Proofs of Section IV-B

Theorem 1 follows from two lemmas about Algorithm 1: it is correct (Lemma 4) and runs in time $O(n \cdot \alpha)$ (Lemma 6). The running-time bound in turn rests on the disjointness of the intervals walked along each $(\mathsf{so} \cup \mathsf{ww})^+$-chain (Lemma 5).

Lemma 4. *On a log-stream L, Algorithm 1 reports a violation upon a transaction t_i iff the prefix $t_\perp, t_1, \ldots, t_i$* does not *satisfy* SI*.*

Proof. As in Lemma 1, the outer loop maintains $currSnap = S_{\mathsf{pred}(t)}$ when t is checked, since ApplyWrites is called after the check; in particular, Δ holds, for the writes of each processed transaction c, the values of $S_{\mathsf{pred}(c)}$ on the keys c writes.

We claim the walk maintains the invariant that, at the top of each iteration,

$$wrongKeys \;=\; \{\, x \mid \mathrm{R}(x,v) \in t \wedge v \neq S_c(x) \,\}.$$

Initially $c = \mathsf{pred}(t)$ and the invariant is the initialization of $wrongKeys$ (line 11). For the step, note that $S_{\mathsf{pred}(c)}$ differs from S_c exactly on the keys written by c, where it holds the values that c overwrote, which is what Δ holds for c's writes; the membership updates, restricted to keys read by t, are precisely the resulting corrections.

Hence the walk checks t against the database states at $\mathsf{pred}(t), \mathsf{pred}(\mathsf{pred}(t)), \ldots$, in order, accepting at the first candidate at which t can take a snapshot and stopping after checking $t_{\mathrm{MAX}}(t)$, whose deltas are recorded since it is processed before t. By Lemma 2, the walk therefore accepts iff t can take a valid snapshot.

Thus, upon t_i, the walks so far have decided the condition of Lemma 2 for every transaction of the prefix, and a violation has been reported iff some transaction of the prefix fails it; by Proposition 1, this is exactly when the prefix does not satisfy SI. □

The running time rests on the following disjointness property.

Lemma 5. *Let $C \subseteq \mathsf{txn}(L)$ be a chain of $(\mathsf{so} \cup \mathsf{ww})^+$. Then the intervals $\mathsf{cand}(t)$ for $t \in C$ are pairwise disjoint.*

Proof. Let $t_1, t_2 \in C$ with $t_1 \xrightarrow{\mathsf{so} \cup \mathsf{ww}}{}^{+} t_2$, and let u be the last transaction before t_2 on a witnessing path, so that $t_1 \xrightarrow{(\mathsf{so} \cup \mathsf{ww})^+}{}^{?} u \xrightarrow{\mathsf{so} \cup \mathsf{ww}} t_2$.

Since u is a direct $\mathsf{so} \cup \mathsf{ww}$ predecessor of t_2, the definition of t_{MAX} gives $u \xrightarrow{\mathsf{co}}{}^{?} t_{\mathrm{MAX}}(t_2)$, and since $(\mathsf{so} \cup \mathsf{ww})^+ \subseteq \mathsf{co}$, also $t_1 \xrightarrow{\mathsf{co}}{}^{?} u$. Now any $t' \in \mathsf{cand}(t_1)$ satisfies $t' \xrightarrow{\mathsf{co}} t_1 \xrightarrow{\mathsf{co}}{}^{?} t_{\mathrm{MAX}}(t_2)$, so $t' \notin \mathsf{cand}(t_2)$. □

Lemma 6. *Given a log-stream L, Algorithm 1 runs in $O(n \cdot \alpha)$ time, where n is the number of operations and α is the width of $(\mathsf{so} \cup \mathsf{ww})^+$.*

Proof. By Dilworth's theorem [47], a partial order of width α can be partitioned into α chains. Therefore, $\mathsf{txn}(L)$ partitions into α chains of $(\mathsf{so} \cup \mathsf{ww})^+$. By Lemma 5, within each chain C the intervals are disjoint, so

$$\sum_{t \in C} \sum_{t' \in \mathsf{cand}(t)} (1 + |t' \cap \mathrm{W}|) \;\leq\; \sum_{t' \in \mathsf{txn}(L)} (1 + |t' \cap \mathrm{W}|) \;\leq\; 2n,$$

and summing over the α chains bounds the double sum of the walk cost by $O(n \cdot \alpha)$. The remaining work is $O(1)$ per operation, i.e., $O(n)$ overall, including the maintenance of $lastTxn$ and $lastWr$ and the computation of each $t_{\mathrm{MAX}}(t)$, which takes the co-latest of the $1 + |t \cap \mathrm{W}|$ transactions $lastTxn[\mathsf{sess}(t)]$ and $lastWr[x]$, for x written by t. □

Lemma 3. *Algorithm 1 runs in $O(n)$ time on log-streams satisfying* SER.

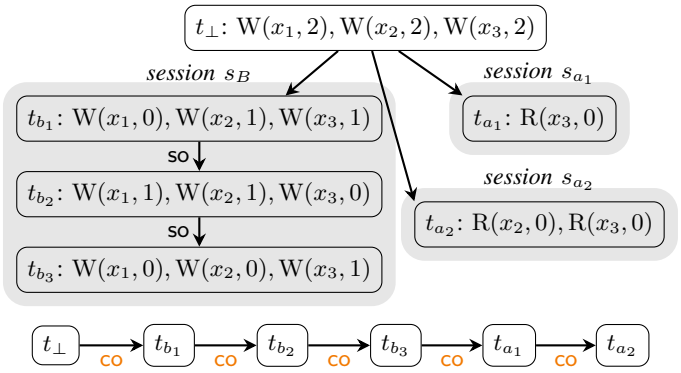


Fig. 12: A log-stream L constructed from the HS instance $A = \{[0,0,1],[0,1,1]\}, B = \{[0,1,1],[1,1,0],[0,0,1]\}$, as described in the proof of Theorem 2, with the co-order of its transactions shown below. L is consistent under SI iff the instance is negative. In this case, L is not consistent, as t_{a_2} cannot take a valid snapshot. Indeed, a_2 hits every $b \in B$, so the HS instance is positive.

Proof. As in Lemma 4, $currSnap = S_{\mathsf{pred}(t)}$ when t is checked, so $wrongKeys$ is initialized to the keys on which t disagrees with the database state $S_{\mathsf{pred}(t)}$.

When L satisfies SER, Lemma 1 gives that every $t \neq t_\perp$ can take a snapshot at $\mathsf{pred}(t)$; hence, $wrongKeys = \emptyset$ and the walk accepts at its newest candidate $\mathsf{pred}(t)$, exactly the SER check. Each walk thus visits a single candidate, so the walk cost of Lemma 6 contributes only $O(1)$ per transaction, and the total running time is $O(n)$. □

C. Proofs of Section IV-C

We provide here the full proof of Theorem 2.

Theorem 2. *For any $0 \leq \lambda \leq 1$, every algorithm (monitoring or not) for deciding the SI-consistency of a log-stream of n operations, m transactions, and $(\mathsf{so} \cup \mathsf{ww})^+$-width $\alpha = \Theta(m^\lambda)$ must take $\Omega(n^{1+\lambda-\epsilon})$ time, for all $\epsilon > 0$, under the HS hypothesis.*

Proof. Given an HS instance $A, B \subseteq \{0,1\}^d$ with $|A| = q^\lambda$ and $|B| = q$ for some q and $0 \leq \lambda \leq 1$, we construct a log-stream over the keys $x_1, \ldots, x_d$, given as the following sequence of transactions. The initial transaction $t_\perp$ writes 2 to every key.

Then, for each $b_i \in B$, one transaction t_{b_i} in a common session s_B, writing $\mathrm{W}(x_j, b_i[j])$ for every $j \in [d]$. Finally, for each $a_i \in A$, one read-only transaction t_{a_i} in its own session s_{a_i}, reading $\mathrm{R}(x_j, 0)$ for every j with $a_i[j] = 1$.

We argue that the log-stream satisfies SI iff every $a \in A$ is orthogonal to some $b \in B$, that is, iff the HS instance is negative. The transactions $t_\perp$ and t_{b_i} read nothing, so only the transactions t_{a_i} constrain satisfaction. Each t_{a_i} is read-only and opens a fresh session, so $t_{\mathrm{MAX}}(t_{a_i}) = t_\perp$ and every

earlier transaction is a candidate. Since each t_{b_i} writes all keys, the database state at t_{b_i} is exactly the vector b_i; the database state at $t_\perp$ is $(2, \ldots, 2)$; and the database state at an earlier $t_{a_{i'}}$ equals that at t_{b_q}, the co-latest writer. Hence t_{a_i} can take a valid snapshot iff some b has $b[j] = 0$ on every j with $a_i[j] = 1$, i.e., iff a_i is orthogonal to some b.

By Lemma 2, the log-stream satisfies SI iff every a_i admits such a b, as claimed.

For the parameters, the construction has $n = \Theta(q \cdot d)$ operations, $m = q^\lambda + q + 1$ transactions, $k = q^\lambda + 1$ sessions, and $\ell = d$ keys, and it is produced in $O(q \cdot d)$ time. The transactions t_{a_i} are pairwise incomparable in $(\mathsf{so} \cup \mathsf{ww})^+$, they write nothing and share no session, so $\alpha = k = \Theta(m^\lambda)$.

By Proposition 1, the monitor's verdict upon the last transaction decides whether the whole log-stream satisfies SI. An algorithm running in $O(n^{1+\lambda-\epsilon})$ total time would thus decide the HS instance in $O((q \cdot d)^{1+\lambda-\epsilon}) = O(q^{1+\lambda-\epsilon} \cdot \mathrm{poly}(d))$ time. This contradicts the unbalanced HS hypothesis for $\lambda_a = \lambda$ and $\lambda_b = 1$. □

D. Proofs of Section IV-D

We prove the results of Section IV-D in the order they are presented.

The memory usage of Algorithm 1. We show that Algorithm 1 uses linear space and demonstrate that the memory optimization described in Section IV-D is sound.

Lemma 7. *Given a log-stream L, Algorithm 1 uses $O(n)$ space, where n is the number of operations.*

Proof. The working memory includes (i) the delta store Δ holding one value per write, (ii) $currSnap$ and $lastWr$ holding one item per key, (iii) $lastTxn$ holding one item per session, and (iv) $rdVal$ and $wrongKeys$ holding at most one item per operation in the transaction. All of these are bounded by $O(n)$. □

Algorithm 2 A memory-optimized monitor for SI

```
1:  procedure MonitorSIOptimized(L)
2:      Lines 5–6 of Algorithm 1
3:      txns ← empty list
4:      for each transaction t streamed by L do
5:          Lines 8–25 of Algorithm 1
6:          if t = t_E(s) for some session s then
7:              Remove s from lastTxn
8:          Add t to txns
9:          while txns is not empty do
10:             t' ← first(txns)
11:             if t' ≠ t_⊥ ∧ sess(t') ∈ lastTxn ∧ t' =
                lastTxn[sess(t')] then
12:                 break
13:             Remove t' from txns
14:             Remove the writes of t' from Δ
```

Algorithm 2 shows how to modify Algorithm 1 to implement the optimization described in Section IV-D. Compared to Algorithm 1, the new algorithm additionally keeps all transactions in a list $txns$, which is used to decide whether an old transaction can be deleted. After processing each transaction as before, the algorithm first removes a session in case of a session end transaction (lines 6–7). Then, on lines 9–14, it iterates $txns$ (oldest first), deleting transactions until it reaches the newest transaction on some session.

The following lemma states that this process exactly implements the aforementioned discarding criteria.

Lemma 8. *After processing a transaction t, Algorithm 2 deletes a non-deleted transaction t' iff all sessions open at t' have executed a transaction after t'.*

Proof. Recall that a session s is open at a transaction t if $t_\mathrm{B}(s) \xrightarrow{\mathsf{co}}{}^? t$ and $t_\mathrm{E}(s) \not\xrightarrow{\mathsf{co}}{}^? t$, so that s is open at $t_\mathrm{B}(s)$ but not at $t_\mathrm{E}(s)$. Since $t_\perp$ is co-before every session begin transaction, no session is open at $t_\perp$. Throughout, we only consider the transactions received so far. We write $s \in lastTxn$ if $lastTxn[s]$ has been assigned by line 24 (in Algorithm 1) and not since removed by line 7; for every other session, $lastTxn$ returns the default value $t_\perp$ of line 6.

The outer loop maintains the invariant that $s \in lastTxn$ iff s is open at t, in which case $lastTxn[s]$ is the co-latest transaction of s. Indeed, line 24 inserts the session of each incoming transaction $t \neq t_\perp$, which is exactly when that session opens or advances, and line 7 removes it upon $t_\mathrm{E}(s)$, which is exactly when it closes.

Call a transaction t'' *pinned* if $t'' \neq t_\perp$, $\mathsf{sess}(t'') \in lastTxn$, and $t'' = lastTxn[\mathsf{sess}(t'')]$, that is, if the session of t'' is open at t and t'' is its co-latest transaction. Transactions enter $txns$ at the back, in co-order, and the loop of line 9 deletes them from the front until it reaches a pinned one. Hence a transaction survives iff it is pinned or co-after a pinned one, and the lemma amounts to the following claim: some $t'' \xrightarrow{\mathsf{co}}{}^? t'$ is pinned iff some session open at t' has executed no transaction after t'.

For the forward direction, let $t'' \xrightarrow{\mathsf{co}}{}^? t'$ be pinned and let $s = \mathsf{sess}(t'')$. Since $t_\mathrm{B}(s)$ is the co-first transaction of s, we have $t_\mathrm{B}(s) \xrightarrow{\mathsf{co}}{}^? t'' \xrightarrow{\mathsf{co}}{}^? t'$; and $t_\mathrm{E}(s)$ has not been received, as s is open at t. So s is open at t'. Moreover, no transaction of s is co-after t': such a transaction would also be co-after t'', contradicting that t'' is the co-latest transaction of s.

For the converse, let s be open at t' with no transaction co-after t'. Then $t_\mathrm{E}(s)$ has not been received, as it would be one such transaction, so s is open at t as well. Its co-latest transaction $lastTxn[s]$ is therefore pinned, and it is $\xrightarrow{\mathsf{co}}{}^? t'$. □

A space lower bound. We give the full proof of Theorem 3, sketched in Section IV-D.

Theorem 3. *There are families of log-streams L, over $k = 2$ sessions, on which any monitor for* SI *must use $\Omega(\nu^\ell)$ space, for ℓ and ν respective keys and values in L.*

Proof. Recall from Definition 4 that a database state $S\colon \mathsf{Key} \to \mathsf{Val}$ is a map from keys to values, and let $S_\perp$ be the database state mapping every key to a reserved value $v_\perp \in \mathsf{Val}$.

Let U be the set of all database states except $S_\perp$, i.e., $|U| = \nu^\ell - 1$. The construction is parameterized by a subset $A \subseteq U$ and a distinguished database state $S^* \in U$. Given such A and S^*, define the log-stream $L(A, S^*)$ as the following sequence of transactions:

(1) the initial transaction $t_\perp$, writing the reserved value $v_\perp$ to every key;
(2) a sequence of $|A|$ transactions on a session s_1: each corresponds to a distinct element $S \in A$, and contains an operation $\mathrm{W}(x, S(x))$ for each $x \in \mathsf{Key}$;
(3) a transaction t_R on a session s_2, containing an operation $\mathrm{R}(x, S^*(x))$ for each $x \in \mathsf{Key}$.

Consider then a run of any monitor on $L(A, S^*)$ at the point where the transaction t_R is processed.

Since the transactions of Items (1) and (2) are write-only, the monitor must, up to this point, report that every prefix satisfies SI, independently of A.

Now, t_R is read-only and opens a fresh session, so $t_{\mathrm{MAX}}(t_\mathrm{R}) = t_\perp$ and every earlier transaction is a candidate; since every transaction writes all keys, the database states at these candidates are exactly $S_\perp$ and the elements of A. As $S^* \neq S_\perp$, the transaction t_R can take a valid snapshot iff $S^* \in A$; by Lemma 2, the log-stream satisfies SI iff $S^* \in A$.

Consequently, for two distinct subsets $A \neq B$ of U and any S^* in the symmetric difference of A and B, the verdict of the monitor on $L(A, S^*)$ and $L(B, S^*)$ must differ. Furthermore, since the verdict is decided only when processing t_R, which does not depend on A or B, the difference in verdict is entirely due to the state of the memory of the monitor.

There are $2^{|U|} = 2^{\nu^\ell - 1}$ subsets of U, so the monitor must reach at least $2^{\nu^\ell - 1}$ distinct memory configurations upon processing t_R. A monitor using σ space admits $2^{O(\sigma)}$ configurations, so $\sigma = \Omega(\nu^\ell)$ on the log-stream of one of these subsets.

Finally, taking $A = U$, the log-stream has $\nu^\ell + 1$ transactions of ℓ operations each, so $n = \ell \cdot (\nu^\ell + 1) \geq \nu^\ell - 1$, and the bound reads $\Omega(\min(n, \nu^\ell))$. □

Figure 13 illustrates the construction of Theorem 3.

An essentially matching upper bound. We show that there is, in fact, a monitor whose worst-case memory usage is not linear in n, but depends on the number of keys and values in the database.

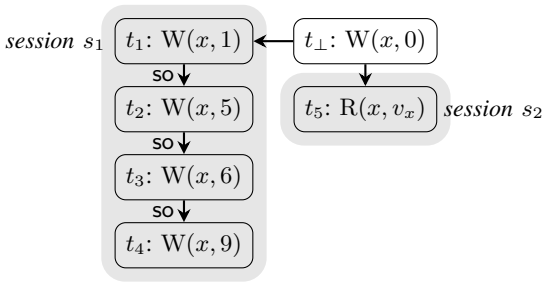


Fig. 13: A log-stream L, with transactions labeled in co-order, exemplifying the construction of Theorem 3. Let the value domain be the natural numbers from 0 to 9. The transactions of session s_1, alongside $t_\perp$, write to x the set of values in $V = \{0, 1, 5, 6, 9\}$. The transaction t_5 can take a valid snapshot if and only if $v_x \in V$, therefore the monitor must "remember" V.

Theorem 4. *Given a database log-stream L, monitoring for* SI *can be solved using $O(\ell \cdot \nu^\ell + k)$ memory and $O(n \cdot \alpha + m \cdot \ell)$ total time.*

We remark that the monitor behind Theorem 4 is primarily of theoretical interest, due to the large ν^ℓ factor in its memory; its significance is that its memory usage (essentially) matches the lower bound of Theorem 3. In practice, Algorithm 1 equipped with the memory-optimization heuristic presented in the beginning of this section performs better. In high level, the monitor modifies Algorithm 1 to store one delta per distinct snapshot, rather than one per transaction. Hence, the worst-case memory usage no longer grows with the number of operations n, but is dominated by the number of different snapshots, of which there are ν^ℓ.

The memory of Algorithm 1 scales with the size of the log-stream, as its delta store grows with n (Lemma 7). We now show that this dependency is avoidable, by a monitor whose memory is bounded by the number of possible database states.

Algorithm 3 keeps a list *reps* of representatives, one per distinct database state, storing a delta per representative rather than the state itself, and walks that list as in Algorithm 1. The list is bounded because there are only ν^ℓ possible database states, and once two candidates carry the same one, checking the co-latest of them suffices. Accordingly, for a transaction t and a database state S occurring at some $t' \in [t_{\mathrm{MAX}}(t), t)$, define the *representative* of S for t as the co-latest transaction t^* before t with $S_{t^*} = S$.

Each representative c in *reps* carries a delta $\Delta[c]$, mapping each key to the value it held at $\mathsf{cpred}(c)$, the predecessor of c in the list. Duplicates are detected with a dictionary *lookup* from database states to their representatives, which can be updated in $O(1)$ per write, with exact comparison of database states on lookup. When the database state at t repeats the one at a representative u, the entries of $\Delta[u]$ are incorporated into the delta of u's successor b, which must now undo past u as

Algorithm 3 A bounded-memory monitor for SI

```
1:  procedure ApplyWrites(t)
2:      Δ[t] ← λx. ⊥
3:      for w = W(x, v) ∈ t ∩ W do
4:          Δ[t][x] ← currSnap[x]          ▷ the value that w overwrites
5:          currSnap[x] ← v
6:  procedure Deduplicate(t)
7:      insert t at the front of reps
8:      if lookup[currSnap] = u for some u then
9:          b ← chain successor of u in reps
10:         Δ[b][x] ← Δ[u][x] for each x ∈ dom(Δ[u])
11:         unlink u from reps
12:     lookup[currSnap] ← t
13: procedure MonitorSICompact(L)
14:     currSnap ← λx. ⊥;   lookup ← ∅
15:     reps ← empty list
16:     lastTxn ← λs. t⊥;   lastWr ← λx. t⊥
17:     for each transaction t streamed by L do
18:         if t ≠ t⊥ then
19:             t_MAX(t) ← co-latest of
                    {lastTxn[sess(t)]} ∪ {lastWr[x] | t writes x}
20:             rdVal ← λx. (v if R(x, v) ∈ t, ⊥ otherwise)
21:             wrongKeys ← {x | R(x, v) ∈ t ∧ v ≠ currSnap[x]}
22:             c ← head of reps
23:             found ← FALSE
24:             while TRUE do
25:                 if wrongKeys = ∅ then
26:                     found ← TRUE; break
27:                 if cpred(c) = ⊥ or cpred(c) -co-> t_MAX(t) then
28:                     break
29:                 for x ∈ dom(Δ[c]) s.t. rdVal[x] ≠ ⊥ do
30:                     if rdVal[x] = Δ[c][x] then
31:                         wrongKeys ← wrongKeys \ {x}
32:                     else
33:                         wrongKeys ← wrongKeys ∪ {x}
34:                 c ← cpred(c)
35:             if found = FALSE then
36:                 report violation
37:         ApplyWrites(t)
38:         if t ≠ t⊥ then lastTxn[sess(t)] ← t
39:         Deduplicate(t)
40:         lastWr[x] ← t for each x written by t
```

well, and u is unlinked; on shared keys the entries of u are maintained, as they undo further back.

We can then execute the walk in the same manner as Algorithm 1, iterating through representatives instead of transactions, which does not compromise correctness.

Proposition 2. *For every transaction $t \in \mathsf{txn}(L) \setminus \{t_\perp\}$ and each candidate transaction $c \in [t_{\text{MAX}}(t), t)$, we have that t can take a snapshot at c iff t can take a snapshot at the representative of S_c for t.*

Proof. Let r be the representative of S_c for t. Since c is itself a transaction before t carrying S_c, co-latestness gives $c \xrightarrow{\text{co}}{}^{?} r \xrightarrow{\text{co}} t$, and $t_{\text{MAX}}(t) \xrightarrow{\text{co}}{}^{?} c$ yields $r \in [t_{\text{MAX}}(t), t)$. Moreover, $S_r = S_c$ by the definition of a representative. By Definition 5, whether t can take a snapshot at a transaction $t'' \xrightarrow{\text{co}} t$ depends only on $S_{t''}$; since $c, r \xrightarrow{\text{co}} t$ and $S_r = S_c$, the transaction t can take a snapshot at c iff it can take a snapshot at r. □

Theorem 4. *Given a database log-stream L, monitoring for SI can be solved using $O(\ell \cdot \nu^\ell + k)$ memory and $O(n \cdot \alpha + m \cdot \ell)$ total time.*

Proof. For correctness, the invariants of Lemma 4 carry over: when t is checked, $currSnap = S_{\mathsf{pred}(t)}$, and $reps$ holds, in co-order, exactly one representative per distinct database state among $S_{t_\perp}, \ldots, S_{\mathsf{pred}(t)}$, each carrying the delta to its predecessor in the list. The folding preserves the latter: after u is unlinked, $\mathsf{cpred}(b)$ is the former $\mathsf{cpred}(u)$, and the merged delta maps each key to the value it held there, as the entries of $\Delta[u]$ are maintained on shared keys. The walk thus checks t against exactly the representatives of $S_{t_{\text{MAX}}(t)}, \ldots, S_{\mathsf{pred}(t)}$ for t, which decides the interval condition of Lemma 2 by Proposition 2. As in Lemma 4, a violation is reported upon t_i iff some transaction of the prefix $t_\perp, t_1, \ldots, t_i$ fails that condition, which by Proposition 1 is exactly when the prefix does not satisfy SI.

For the running time, lookups, insertions, and delta merges cost $O(\ell)$ per transaction, for the $O(m \cdot \ell)$ term. The walk for t visits only representatives in $[t_{\text{MAX}}(t), t)$ and scans, per representative, one delta entry per key written by the transactions it absorbed, all within the interval; the accounting of Lemma 6 therefore applies unchanged, for the $O(n \cdot \alpha)$ term.

Each walk visits at most ν^ℓ representatives, as the entries of $reps$ carry pairwise distinct database states, so the walk cost is in fact $O(n \cdot \min(\alpha, \nu^\ell))$.

For the memory, the delta store is now indexed by representatives instead of writes, and $reps$ holds one entry of size $O(\ell)$ per distinct database state, hence at most ν^ℓ of them; the remaining structures are as in Lemma 7, namely $currSnap$ and $lastWr$ ($O(\ell)$) and $lastTxn$ ($O(k)$). □